

Improved Subfield Curve Search For Specific Field Characteristics

Jesús-Javier Chi-Domínguez 

Cryptography Research Center, Technology Innovation Institute, Abu Dhabi, UAE

Abstract. Isogeny-based cryptography relies its security on the hardness of the supersingular isogeny problem: finding an isogeny between two supersingular curves defined over a quadratic field extension of $\mathbb{F}_p$.

The Delfs-Galbraith algorithm is one of the most efficient procedures for solving the supersingular isogeny problem with a time complexity of $\tilde{O}(p^{1/2})$ operations. The bottleneck of the Delfs-Galbraith algorithm is the so-called subfield curve search (i.e., finding an isogenous supersingular elliptic curve defined over the prime field), which determines the time complexity of the aforementioned algorithm.

Given that, for efficiency, most recent isogeny-based constructions propose using finite fields with field characteristics equal to $p = 2^a \cdot f - 1$ for some positive integers a and f . This work primary focuses on primes of that particular form and presents two heuristic algorithms for finding subfield curves with a time complexity of $\mathcal{O}(p^{1/2})$ operations and a memory complexity polynomial in $\log_2 p$. We show how to adapt these algorithms to primes of the form $p = d^a \cdot f - 1$ with d being a small integer, with a particular focus on $d = 12$. We provide concrete time-complexity bounds for both kinds of primes $p = 2^a \cdot f - 1$ and $p = 12^a \cdot f - 1$. Our algorithms exploit the existence of large d^a -torsion points and extend the subfield root detection algorithm of Corte-Real Santos, Costello, and Shi (Crypto, 2022) to a projective scenario where one can work with the curve coefficients instead of their explicit j -invariants.

When considering the bit complexity $\mathcal{O}((\log_2 p)^\kappa p^{1/2})$ of the subfield curve searching algorithms, our results suggest a smaller value of $\kappa = \log_3 5 \approx 1.465$ compared to the state of the art value of $\kappa = 2$. We discuss how this lower bit complexity affects current isogeny-based constructions.

We highlight that our algorithms easily extend to primes of the form $p = 2^a \cdot f + 1$, $p = 4 \cdot d_1 \cdot d_2 \cdots d_n - 1$ for small odd primes d_i 's, and primes such that $p - 1$ and $p + 1$ are B -smooth for some integer B .

Keywords: Cryptanalysis · Delfs-Galbraith Algorithm · Isogeny-based Cryptography · Subfield Curve Search · Supersingular Isogeny Problem

1 Introduction

The security of most of the isogeny-based cryptographic schemes relies on the hardness of the *supersingular isogeny problem*, which refers to finding an isogeny $\phi: \mathcal{E} \rightarrow \mathcal{F}$ between two given supersingular elliptic curves defined over $\mathbb{F}_{p^2}$.

The best classical algorithm for solving the supersingular isogeny problem is the Delfs-Galbraith algorithm [DG16], which splits into two main steps:

1. *Subfield curve search:* Fix an integer ℓ , and find two $\ell^\bullet$ -isogenies, $\phi_0: \mathcal{E} \rightarrow \mathcal{E}'$ and $\phi_1: \mathcal{F} \rightarrow \mathcal{F}'$, so the codomain curves $\mathcal{E}'$ and $\mathcal{F}'$ are defined over the subfield $\mathbb{F}_p$.¹ This step runs in $\tilde{O}(p^{1/2})$ field operations.

E-mail: jesus.dominguez@tii.ae (Jesús-Javier Chi-Domínguez)

¹The notation “ $\ell^\bullet$ -isogeny” refers to a ℓ^e -isogeny for some integer number e .



2. *Subfield isogeny search*: Find an isogeny $\psi: \mathcal{E}' \rightarrow \mathcal{F}'$ between $\mathcal{E}$ and $\mathcal{F}'$ defined over $\mathbb{F}_p$. This step runs in $\tilde{\mathcal{O}}(p^{1/4})$ field operations.

The Delfs-Galbraith algorithm has a running time of $\tilde{\mathcal{O}}(p^{1/2})$ field operations, and its bottleneck step is the subfield curve search. Hence, any improvement in the subfield curve search of the Delfs-Galbraith algorithm would impact the security of most isogeny-based constructions.

Motivation. Given that the shape of the field characteristic p impacts the field arithmetic and performance of isogeny calculations, and it does not affect security, recent works push for primes of the form $p = 2^a \cdot f - 1$. For example,

- SQIsign v1.0 [CSCD⁺23] requires primes $p = (4 \cdot 3)^a \cdot f - 1$ with $f \in \mathbb{N}$ and $a \approx \frac{1}{2} \log_{12} p$.
- SQIsignHD [DLRW24] suggests primes $p = 2^a \cdot 3^b \cdot f - 1$ with small $f \in \mathbb{N}$, $a \approx \frac{1}{2} \log_2 p$, and $b \approx \frac{1}{2} \log_3 p$.
- ApresSQI [SEMR24] works over primes of the form $p = 2^a \cdot f - 1$ with $f \in \mathbb{N}$ and $a \approx \frac{1}{2} \log_2 p$.
- SQIsign2D-West [BDD⁺24], SQIPrime [DF24], SQIsign2D-East [NOC⁺24], and SQIsign v2.0 [AAA⁺25] ask for $p = 2^a \cdot f - 1$ with small $f \in \mathbb{N}$ and $a \approx \log_2 p$.

All the above prime numbers determine an interesting structure in the $(p+1)$ -torsion subgroup of the curves: there are points of large torsion 2^a . To our knowledge, the Delfs-Galbraith algorithm does not exploit the structure of the torsion subgroup of the curve; it performs isogeny walks through modular polynomials at the cost of field exponentiations.

Our Contribution. We show that the existence of a large torsion- 2^a subgroup reduces the complexity of the Delfs-Galbraith algorithm from $\tilde{\mathcal{O}}(p^{1/2})$ to $\mathcal{O}(p^{1/2})$ field operations.

Firstly, we propose a dedicated and optimized subfield search through a Depth-First-Search traversal of the 2-isogeny tree over specific primes, $p = 2^a \cdot f - 1$. Our algorithm performs the subfield search by calculating 2-isogenies from kernel generators, and therefore, it operates on the curves instead of the j-invariants. Such an approach does not require the calculation of the expensive square roots as in the original Delfs-Galbraith algorithm. Next, we extend our algorithm to the setting $p = d^a \cdot f - 1$ for some $a, d, f \in \mathbb{N}$, with a particular focus on $d = 12$.

Secondly, we extend the subfield root detection algorithm from [SCS22] to our case study. Given that our proposed algorithm does not operate on the j-invariants of the curves but on the curves themselves, it implies we cannot directly use the subfield root detection algorithm of [SCS22] without calculating a field inversion. However, we show how to extend that subfield root detection algorithm when we only have the denominator and numerator of the j-invariants of the curves (that is, without requiring any field inversion).

In a nutshell, we propose two new algorithms, Searcher_d and SuperSearcher_d , with a time complexity equal to

$$c \cdot \frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} = \mathcal{O}\left(p^{1/2}\right)$$

and a memory complexity polynomial in $\log_2 p$, where $\#\mathbb{S}_{p^2} \approx \frac{1}{12}p$ and $\#\mathbb{S}_p = \tilde{\mathcal{O}}(p^{1/2})$ determine the number of supersingular elliptic curves with j-invariants defined over $\mathbb{F}_{p^2}$ and $\mathbb{F}_p$, respectively. Additionally, we provide a tighter complexity analysis of the algorithm in [SCS22], which includes the hidden logarithm factor in the complexity analysis. We present explicit time complexities regarding field operations through x -only points on the Kummer lines and validate our claims through extensive experiments (see Table 1). We provide a proof-of-concept implementation using SageMath [The25] available at [Com26].

Table 1: The constants $0.2 \lesssim \gamma_2 \lesssim 1.64$ and $0.07 \lesssim \gamma_1 \lesssim 0.51$ are as detailed in Section 3. The row concerning **SuperSolver** corresponds with our analysis on the algorithm **SuperSolver** from [SCS22]. Here $n = \frac{1}{2} \log_2 p$. The last column presents the maximum number of inspected curves for each algorithm.

Algorithm	Running Time		Number of Inspected Curves
	Average	Maximum	
SuperSolver	$\gamma_1 \cdot \log_2 p \cdot 2^n$	$\gamma_2 \cdot \log_2 p \cdot 2^n$	$1.438 \cdot 2^n$
Searcher ₂	$33 \cdot 2^n$	$66 \cdot 2^n$	2^n
Searcher ₄	$30.75 \cdot 2^n$	$82 \cdot 2^n$	$1.333 \cdot 2^n$
Searcher ₁₂	$58.21 \cdot 2^n$	$127 \cdot 2^n$	$1.091 \cdot 2^n$
SuperSearcher ₂	$20 \cdot 2^n$	$65 \cdot 2^n$	$1.625 \cdot 2^n$
SuperSearcher ₄	$19.83 \cdot 2^n$	$42.96 \cdot 2^n$	$1.083 \cdot 2^n$
SuperSearcher ₁₂	$19.75 \cdot 2^n$	$39.49 \cdot 2^n$	2^n

We highlight that our results should be taken as asymptotic improvements: in the matter that the bitlength of p increases, the bigger improvements for the subfield curve searching.

Our results should be taken as asymptotic improvements: in the matter that the bitlength of p increases, the bigger improvements for the subfield curve searching. For example, Table 6 from [SCS22] presents values of c ranging from $c = 35$ (primes of 50 bits) to $c = 90$ (primes of 800 bits), while for **SuperSearcher** we have a fixed value of $c = 20$. In particular, our results should be taken as cautionary advice for proposing conservative parameter sets in isogeny-based constructions with 2λ -bits prime numbers of the form $p = 2^a \cdot f - 1$ with $a \geq \lambda$ and $\lambda \in \{128, 192, 256\}$. For instance, most isogeny-based constructions propose parameter sets that take into account the asymptotic bit complexity $\mathcal{O}((\log_2 p)^\kappa p^{1/2})$, without considering any constant factors and taking the value of $\kappa = 2$ from [EHMP20]. This consideration is, for example, presented in **SQIsign** v2.0 [AAA⁺25, Section 10.2.1] and aligned to the NIST security considerations of [Nat22, Section 4.B.3]. However, our results suggest a smaller value of $\kappa = \log_3 5 \approx 1.465$ compared to the state of the art, for which our results suggest 5.8, 2.9 and 6.3 bits below the NIST security requirements for Level 1, 3 and 5, respectively. In summary, our algorithms save a factor of $\log_2 p$ compared to the **SuperSolver** algorithm of [SCS22], and around $\frac{1}{2} \log_2 p$ compared to [EHMP20].

We stress that our results easily extend to more generic primes like $p = 2^a \cdot f + 1$, $p = d^a \cdot f - 1$, $p = 4 \cdot d_1 \cdot d_2 \cdots d_n - 1$ for small odd primes d_i 's, and primes such that $p - 1$ and $p + 1$ are B -smooth for some integer B . We provide a small discussion on that families of primes in the Appendix.

Organization. Section 2 provides the preliminaries. Section 3 details and refines an analysis of the running time of **SuperSolver**. Section 4 describes the two algorithms, **Searcher_d** and **SuperSearcher_d** with special focus on $d = 2, 4, 12$; in particular, Section 4.3 presents our projective fast subfield root detection. We draw our experiments in Section 5 and discuss in Section 6 some implications of our work to the state-of-the-art constructions.

2 Preliminaries

Through this paper, we denote the multiplications, squares, and additions in $\mathbb{F}_{p^2}$ by **M**, **S**, and **A**, respectively. On the other hand, **mul**, **sqr**, and **add** denote multiplications, squares,

and additions in $\mathbb{F}_p$.² For any $x \in \mathbb{R}$, we denote the nearest integer to x by $\lfloor x \rfloor$.

Let $p = 2^a \cdot f - 1$ be a prime number of r -bits such that $p \equiv 3 \pmod{4}$ and $a \geq \frac{1}{2}r$. Let $\mathbb{F}_{p^2} = \mathbb{F}_p[i]/(i^2 + 1)$ be a quadratic field extension of $\mathbb{F}_p$ and $\mathcal{E}$ be supersingular elliptic curves with $\#\mathcal{E}(\mathbb{F}_{p^2}) = (p \pm 1)^2$ points. In particular, we assume $\mathcal{E}$ is given in Montgomery form (i.e., $\mathcal{E}: y^2 = x^3 + Ax^2 + x$ for some $A \in \mathbb{F}_{p^2}$). The j -invariant of $\mathcal{E}$ is $j(\mathcal{E}) = 256(A^2 - 3)^3/(A^2 - 4)$, and two elliptic curves $\mathcal{E}$ and $\mathcal{F}$ are isomorphic iff $j(\mathcal{E}) = j(\mathcal{F})$. We use the term “subfield curve” to mean that $j(\mathcal{E})$ belongs to $\mathbb{F}_p$.

An isogeny $\phi: \mathcal{E} \rightarrow \mathcal{F}$ is a non-constant morphism between two elliptic curves such that $\#\ker \phi < \infty$ and $\phi(\mathbf{0}_{\mathcal{E}}) = \mathbf{0}_{\mathcal{F}}$, where $\mathbf{0}_{\mathcal{E}}$ (resp. $\mathbf{0}_{\mathcal{F}}$) denotes the point at infinity on $\mathcal{E}$ (resp. $\mathcal{F}$). We say that ϕ is an ℓ -isogeny if $\ker \phi$ has size ℓ . The dual of ϕ is another ℓ -isogeny $\hat{\phi}: \mathcal{F} \rightarrow \mathcal{E}$ such that $\phi \circ \hat{\phi} = [\ell] = \hat{\phi} \circ \phi$, where $[\ell]: P \mapsto [\ell]P$ denotes the multiplication-by- ℓ map. There are exactly $(\ell + 1)$ ℓ -isogenies (up to isomorphism) with domain $\mathcal{F}$, one of them corresponding with $\hat{\phi}: \mathcal{F} \rightarrow \mathcal{E}$.

We consider the supersingular isogeny graph $\mathcal{X}(\overline{\mathbb{F}}_p, \ell)$ as the directed graph where the vertices are the j -invariants of supersingular elliptic curves defined over $\overline{\mathbb{F}}_p$, and the edges are ℓ -isogenies. However, we only consider curves defined over $\mathbb{F}_{p^2}$ since the j -invariants of supersingular elliptic curves always belong to $\mathbb{F}_{p^2}$. The graph $\mathcal{X}(\overline{\mathbb{F}}_p, \ell)$ is fully connected for every prime ℓ [Koh96], and it is an expander graph [CLG09]. In particular, the set of all j -invariants of supersingular elliptic curves has cardinality $\#\mathbb{S}_{p^2} \approx \frac{1}{12}p$, and only $\#\mathbb{S}_p = \tilde{O}(p^{1/2})$ of them belong to $\mathbb{F}_p$ [DG16]. Since the expected time for a random walk on a set of cardinality $\#\mathbb{S}_{p^2}$ to reach a subset of size $\#\mathbb{S}_p$ is $\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p}$, we consider the following assumption, which plays an important role in subsequent sections.

Assumption 1. Consider two supersingular elliptic curves $\mathcal{E}_0$ and $\mathcal{E}_1$ defined over $\mathbb{F}_{p^2}$, connected by an unknown random N -isogeny $\phi: \mathcal{E}_0 \rightarrow \mathcal{E}_1$ for some integer $N \geq p^{1/2}$. In particular, assume $\mathcal{E}_0: y^2 = x^3 + x$. Let $d \in \mathbb{N}$ and L be a list of small positive integers such that $d \notin L$, and let h be the smallest positive integer such that $h \leq \frac{1}{2} \log_d p$ and the set

$$\mathcal{S}_{L,d,h}(\mathcal{E}_1) = \{j(\mathcal{F}) \in \mathbb{S}_{p^2} : \exists \text{ an } M\text{-isogeny } \psi: \mathcal{E}_1 \rightarrow \mathcal{F} \text{ with } M \mid d^h \prod_{\ell \in L} \ell\}$$

has cardinality

$$\#\mathcal{S}_{L,d,h}(\mathcal{E}_1) \approx \frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p}.$$

Then, there exists at least one element $j(\mathcal{F})$ in $\mathcal{S}_{L,d,h}(\mathcal{E}_1)$ that belongs to $\mathbb{S}_p$ with overwhelming probability.

To gain more confidence in Assumption 1, we highlight the following three points.

1. The authors in [DG16] argue that for a given supersingular elliptic curve $\mathcal{E}$ defined over $\mathbb{F}_{p^2}$, is expected to exist a short 2-isogeny path of length

$$h = \log_2 \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right) \approx \frac{1}{2} \log_2 p \quad (1)$$

from $\mathcal{E}$ to a subfield curve $\mathcal{F}$. This case corresponds to Assumption 1 with $d = 2$ and $L = 1$.

2. The authors in [SCS22] improve [DG16] and illustrate an isogeny path of length shorter than h by extending the subfield curve search to each ℓ -isogeny with $\ell \in L$. This case corresponds to Assumption 1 with $d = 2$ and L being a list of small integers.
3. Our experiments in the following sections (i.e., Section 5) show that the probability that a subfield curve belongs to $\mathcal{S}_{L,d,h}(\mathcal{E}_1)$ approaches 1 as p increases, thereby strengthening Assumption 1.

²If $p \equiv 3 \pmod{4}$ then we have $M = 3\text{mul} + 5\text{add}$ and $S = 2\text{mul} + 3\text{add}$.

Modular polynomials. The ℓ^{th} modular polynomial $\Phi_\ell(x, y) \in \mathbb{Z}[x, y]$ is an irreducible symmetric polynomial such that $\Phi_\ell(j(\mathcal{E}), j(\mathcal{F})) = 0$ for any pair of two ℓ -isogenous curves $\mathcal{E}$ and $\mathcal{F}$. If ℓ is a prime number, and $j = j(\mathcal{E})$ is the j -invariant of an elliptic curve $\mathcal{E}$ then $\Phi_\ell(x, j) \in \mathbb{F}_{p^2}[x]$ is a polynomial of degree $d_\ell = (\ell + 1)$.

Kummer line. The Kummer line of $\mathcal{E}$ is $\mathcal{E}/\langle\ominus\rangle \cong \mathbb{P}^1$, where $\ominus: P \mapsto -P$ denotes the negation map. A point $x(P)$ on $\mathcal{E}/\langle\ominus\rangle$ is of the form $(X: Z)$ for some $X, Z \in \mathbb{F}_{p^2}$. In particular,

$$x(P) = \begin{cases} (1: 0) & \text{if } P = \mathbf{0}_{\mathcal{E}} \\ (X: Z) & \text{otherwise.} \end{cases}$$

If $x(P) = (X: Z) \neq (1: 0)$ then $x_P = X/Z$ determines the x -coordinate of the point P .

Given the three points $x(P)$, $x(Q)$, and $x(P - Q)$ on $\mathcal{E}/\langle\ominus\rangle$, one can efficiently calculate $x(P + Q)$ and $x([2]P)$ at the cost of $4M + 2S + 6A$ and $4M + 2S + 4A$ operations, respectively [Mon87]. In addition, computing $x([3]P)$ requires $7M + 5S + 10A$ [CH17].

Similarly, ℓ -isogenies on $\mathcal{E}$ nicely restrict to $\mathcal{E}/\langle\ominus\rangle$ [CH17, Ren18, BDLS20]; that is, given a point P on $\mathcal{E}$ and an ℓ -isogeny $\phi: \mathcal{E} \rightarrow \mathcal{F}$ with $\ker \phi = \langle K \rangle$, then $x(\phi(P))$ only depends on $x(P)$, $x([j]K)$ for some integers $j \in [1, \ell] \cap \mathbb{Z}$, and on the Montgomery A -coefficient of $\mathcal{E}$. Table 2 lists the costs for computing codomain curves and pushing points through ℓ -isogenies with $\ell = 2, 3, 4$.

Table 2: The 2-isogeny and 4-isogeny formulas from [Ren18] assume isogeny kernels different from $(0, 0)$. The 3-isogeny cost comes from [CH17].

	2-isogeny		4-isogeny		3-isogeny	
	Codomain	Evaluate	Codomain	Evaluate	Codomain	Evaluate
Cost:	2S	4M	4S	6M + 2S	2M + 3S	4M + 2S

Through the manuscript, we only consider multiplications and squares in the time complexities of the algorithms, we omit additions and subtractions. And, we say that an algorithm succeeds whenever it returns a valid solution (i.e., it does not return $\perp$).

3 On the Delfs-Galbraith Subfield Searching Algorithm

Corte-Real Santos, Costello, and Shi provide an efficient procedure for determining whether an ℓ -isogeny connects a given elliptic curve to a subfield curve [SCS22]. Or, put differently, they describe an efficient algorithm for inspecting all the ℓ -isogenous curves to a given arbitrary supersingular curve. More precisely, they propose a subfield root detection algorithm `NeighbourInFp`(ℓ, j), which determines whether the polynomial $\Phi_\ell(x, j)$ has roots in the field $\mathbb{F}_p$; we give a high-level description of `NeighbourInFp` in Figure 1. The main ingredients of `NeighbourInFp` include i) an inverse-free GCD procedure and ii) an efficient and explicit description of the polynomials g_1 and g_2 , respectively.³ In addition, they show that the cost of `EvaluateModularPolynomial`(ℓ, j) is bounded by $9d_\ell(d_\ell - 1)\text{mul}$ operations, while `InverseFreeGCD`(g_1, g_2) by

$$\frac{1}{2}(\deg g_1 + \deg g_2 + 2)(\deg g_1 + \deg g_2 + 3) - 6$$

field multiplications in $\mathbb{F}_p$. Hence, `NeighbourInFp`(ℓ, j) takes $(11d_\ell^2 - 6d_\ell - 5)\text{mul}$ operations.

³For instance, we have $\deg g_1 = d_\ell$ and $\deg g_2 = d_\ell - 1$, and the leading coefficient of g_1 is equal to one.

⁴We use the inequality $\deg r > 1$ instead of $\deg r \geq 1$ in Figure 1. In [SCS22, Alg. 1], the authors wrote $\deg r \geq 1$ while in their code they used the inequality $\deg r > 1$.

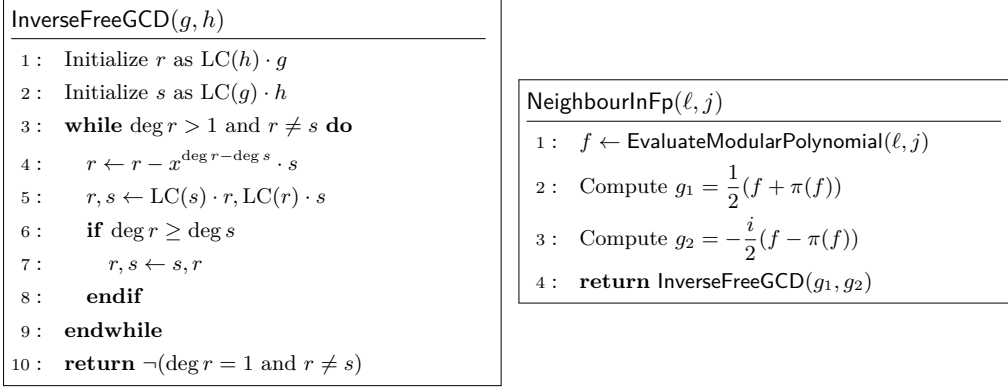


Figure 1: Description of the subfield root detection algorithm from [SCS22]. The inputs ℓ and j of $\text{EvaluateModularPolynomial}$ must satisfies that $j(\mathcal{E}) = j$ for some supersingular elliptic curve $\mathcal{E}$ defined over $\mathbb{F}_{p^2}$. The subroutine $\text{EvaluateModularPolynomial}(\ell, j)$ is as described in [SCS22]: it evaluates $\Phi_\ell(x, y)$ at $y = j$ by calculating $j^2, \dots, j^{d_\ell}$ and multiplying them by the corresponding coefficients of $\Phi_\ell(x, y)$. The inputs g and h of InverseFreeGCD correspond with polynomials $g, h \in \mathbb{F}_{p^2}[x]$ such that $\deg g \geq \deg h$. Here, π denotes the p -th power Frobenius endomorphism on $\mathbb{F}_{p^2}$.⁴

3.1 Overall description of SuperSolver algorithm

As the main application of NeighbourInFp , Corte-Real Santos, Costello, and Shi propose using it in the Delfs-Galbraith Subfield Searching Algorithm. Following the notation in [SCS22], we give in Figure 2 a high-level description of such an optimized subfield-searching algorithm, **SuperSolver**. The time complexity of finding a subfield curve through **SuperSolver** is $\tilde{O}(p^{1/2})$ field operations with a memory complexity polynomial in $\log_2 p$.

3.2 On the upper bound and expected average cost of SuperSolver

To our knowledge, [SCS22] only provides an experimental average cost for **SuperSolver** of

$$58 \cdot \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right)$$

field operations, for 256-bit primes. However, as already highlighted, there is a hidden logarithm factor in the complexity of the Delfs-Galbraith Subfield Searching Algorithm determined by Line 14 in **SuperSolver**, which requires a square root calculation. Given that, we next describe an explicit upper bound on the cost of **SuperSolver**, including the logarithmic factor (this explicit runtime will be the baseline for comparing our proposed algorithms in Section 4).

Cost metric for NeighbourInFp: We use the subfield cost for Line 9 in the **SuperSolver** algorithm as in [SCS22]; that is, we take that cost (per ℓ) as

$$\text{cost}_\ell = \frac{1}{d_\ell} (\text{cost of NeighbourInFp}) \leq \frac{1}{d_\ell} (11d_\ell^2 - 6d_\ell - 5) \text{mul.}$$

Cost metric for solving $\frac{\Phi_2(j(\mathcal{E}_b), x)}{x - j(\mathcal{F}_b)}$: Line 14 in the **SuperSolver** algorithm requires solving a quadratic equation, which summarizes as calculating

$$j(\mathcal{E}_{bi}) = \frac{1}{2} (j(\mathcal{E}_b)^2 - 1488j(\mathcal{E}_b) - j(\mathcal{F}_b) + 162000 \pm \sqrt{\rho_i}) \quad (2)$$

SuperSolver($\mathcal{E}, h, L$)

```

1 : Initialize an empty stack for storage of nodes,  $S$ 
2 : Deterministically compute a 2-isogeny with domain  $\mathcal{E}$  and codomain  $\mathcal{F}$ 
3 : Push the root  $\mathbf{z}_\emptyset = \{j(\mathcal{E}), j(\mathcal{F}), \emptyset\}$  onto  $S$ 
4 : Pop the first element in  $S$ ,  $\mathbf{z}_b = \{j(\mathcal{E}_b), j(\mathcal{F}_b), \mathbf{b}\}$  where  $\mathbf{b} \in \{0, 1\}^n$ 
5 : if  $j(\mathcal{E}_b) \in \mathbb{F}_p$ 
6 :   return  $(\mathbf{b}, 2)$ 
7 : endif
8 : for each  $\ell \in L$ 
9 :   if NeighbourInFp( $\ell, j(\mathcal{E}_b)$ )
10 :    return  $(\mathbf{b}, \ell)$ 
11 :   endif
12 : endfor
13 : if  $n \leq (h - 1)$ 
14 :   Compute the two roots  $j(\mathcal{E}_{b0})$  and  $j(\mathcal{E}_{b1})$  of the polynomial  $\frac{\Phi_2(j(\mathcal{E}_b), x)}{x - j(\mathcal{F}_b)}$ 
15 :   Push  $\mathbf{z}_{b1} = \{j(\mathcal{E}_{b1}), j(\mathcal{E}_b), \mathbf{b}1\}$  onto  $S$ 
16 :   Push  $\mathbf{z}_{b0} = \{j(\mathcal{E}_{b0}), j(\mathcal{E}_b), \mathbf{b}0\}$  onto  $S$ 
17 :   Go to Line 4
18 : endif
19 : if  $S$  is empty
20 :   return  $\perp$ 
21 : else
22 :   Go to Line 4
23 : endif

```

Figure 2: Description of the optimized subfield curve search algorithm from [SCS22]. The input must satisfy $\mathcal{E}$ is a supersingular elliptic curve defined over $\mathbb{F}_{p^2}$, h is a positive integer, and L is a list of positive integers. In practice, L is a list of small (odd prime) numbers. The integer n determines the length of the bitstring $\mathbf{b}$. For the empty bitstring $\mathbf{b} = \emptyset$, we set $n = 0$. In particular, the value of $\mathbf{b}$ describes the 2-isogeny path of length n going from $\mathcal{E} = \mathcal{E}_\emptyset$ to $\mathcal{E}_n$. For each $i = 0, 1$, the node $\mathbf{z}_{bi} = \{j(\mathcal{E}_{bi}), j(\mathcal{F}_{bi}), \mathbf{b}i\}$ satisfies $\mathcal{E}_{bi}$ and $\mathcal{F}_{bi}$ are connected by a 2-isogeny.

where

$$\begin{aligned} \rho_i = & j(\mathcal{E}_b)^4 - 2976j(\mathcal{E}_b)^3 + 2j(\mathcal{E}_b)^2j(\mathcal{F}_b) + 2532192j(\mathcal{E}_b)^2 - 2976j(\mathcal{E}_b)j(\mathcal{F}_b) \\ & - 645205500j(\mathcal{E}_b) - 3j(\mathcal{F}_b)^2 + 324000j(\mathcal{F}_b) - 8748000000. \end{aligned}$$

Using Scott's fast square root method [Sco20], we observe that calculating $\sqrt{\rho_i}$ takes two multiplications and two exponentiations over $\mathbb{F}_p$; in particular, it requires raising to the powers of

$$N = \frac{1}{4}(p+1) \quad \text{and} \quad M = \frac{1}{4}(p-3).$$

In practice, for concrete cryptographic parameters, both N and M have approximately $\frac{1}{2}(r-2)$ as Hamming weight, respectively. For simplicity, we conservatively opt to use the Left-to-right binary algorithm for the field exponentiation. We highlight that we could further optimize such exponentiation by employing the wNAF method or a shorter addition chain. However, this improvement would have only a tiny impact on our purposes and comparisons. Hence,

- Calculating $z^{(p+1)/4}$ takes $\frac{1}{2}(r-2)\text{mul} + (r-2)\text{sqr}$ operations, and
- Computing $z^{(p-1)/3}$ requires $\frac{1}{2}(r-2)\text{mul} + (r-2)\text{sqr}$ operations.

Therefore, the cost of calculating Eq. (2) becomes

$$\begin{aligned} \text{cost}_{\text{eqn}} &= (2\text{mul} + 3\text{sqr}) + (r-2)\text{mul} + 2(r-2)\text{sqr} + (3M + 3S) \\ &= (r+15)\text{mul} + (2r-1)\text{sqr} \approx (3r+14)\text{mul}. \end{aligned} \quad (3)$$

3.2.1 On the explicit running time of SuperSolver:

We describe below a different analysis approach from [SCS22], enabling us to isolate the constant factor in the complexity of the algorithm **SuperSolver**, thus allowing us to describe the running time of **SuperSolver** explicitly.

Recall, **NeighbourInFp** allows us to detect if there is one ℓ -isogenous subfield curve among the $(\ell+1)$ possibilities. That optimization implies an increased number of inspected curves by a factor of $(1 + \sum_{\ell \in L} d_\ell)$ times bigger. Therefore, under Assumption 1, we can decrease the value of h to (at most)

$$h = \log_2 \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right) - \left\lceil \log_2 \left(1 + \sum_{\ell \in L} d_\ell \right) \right\rceil$$

and still inspect around $\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p}$ curves. We thus consider the running time of algorithm **SuperSolver** described in Figure 2 to be bounded by

$$\text{runtime}_1 = \underbrace{\frac{1}{2^{\left\lceil \log_2 \left(1 + \sum_{\ell \in L} d_\ell \right) \right\rceil}}}_{\gamma_p(L)} \left(\text{cost}_{\text{eqn}} + 2 \cdot \sum_{\ell \in L} \overbrace{d_\ell \cdot \text{cost}_\ell}^{\text{NeighbourInFp}} \right) \cdot \frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \text{ mul}. \quad (4)$$

We take advantage of the following three main observations related to function $\gamma_p(L)$:

1. $\sum_{\ell \in L} d_\ell$ is linear in the variables $\ell_1, \dots, \ell_t$.
2. $\sum_{\ell \in L} d_\ell \cdot \text{cost}_\ell$ is quadratic in the variables $\ell_1, \dots, \ell_t$.
3. Function $\gamma_p(L)$ is polynomial in $\ell_1, \dots, \ell_t$, and $\log_2 p$.

Given that the exponential factor

$$\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p}$$

in the running time does not change from the choice of L , we focus on minimizing the value of $\frac{1}{\log_2 p} \cdot \gamma_p(L) \leq \frac{\text{cost}_{\text{eqn}}}{\log_2 p} \in O(1)$. In particular, we use the concrete costs of cost_ℓ from [SCS22] to select the best candidate of L with at most thirteen elements (any other larger prime number ℓ implies a higher number of field operations when inspecting $\mathcal{X}(\overline{\mathbb{F}}_p, \ell)$). More precisely, we perform a brute-force search on $\mathbf{b} \in \{0, 1\}^{13}$ to determine

$$\underset{\mathbf{b}}{\text{argmin}} \frac{1}{\log_2 p} \cdot \gamma_p(L_{\mathbf{b}}),$$

where $L_{\mathbf{b}}$ denotes the list having the i^{th} prime ℓ_i if the i^{th} bit of $\mathbf{b}$ is equal to one. Our experiments illustrate that the running time of **SuperSolver** is bounded by

$$\text{runtime}_1 = \gamma \cdot \log_2 p \cdot 2^h \text{ mul} = \gamma \cdot \log_2 p \cdot \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right) \text{ mul} \quad (5)$$

where $0 < \gamma \lesssim 1.64$.

3.2.2 On the expected average running time of SuperSolver:

Observe that our description of the algorithm SuperSolver in Figure 2 inspects

$$2^{h+1} \cdot \left(1 + \sum_{\ell \in L} d_\ell\right)$$

curves, where $h = \log_2 \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right) - \lfloor \log_2 (1 + \sum_{\ell \in L} d_\ell) \rfloor$.

Remark 1. We stress that under Assumption 1, our description of SuperSolver in Figure 2 is expected to return a subfield curve; however, in the worst case, the algorithm in Figure 2 inspects a total of $2^{h+1} \cdot (1 + \sum_{\ell \in L} d_\ell)$ curves. Therefore, since in the best-case scenario the input curve is a subfield curve, on average we would expect to inspect half of the curves to reach a subfield curve.

Hence, given that we only need to inspect at most $\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p}$ curves to find a subfield curve, the expected running time of the algorithm SuperSolver turns out to be

$$\frac{1}{2} \cdot \left(\frac{2^{\lfloor \log_2 (1 + \sum_{\ell \in L} d_\ell) \rfloor}}{2 \cdot (1 + \sum_{\ell \in L} d_\ell)} \cdot \text{runtime}_1 \right) = \left(\frac{2^{\lfloor \log_2 (1 + \sum_{\ell \in L} d_\ell) \rfloor}}{4 \cdot (1 + \sum_{\ell \in L} d_\ell)} \cdot \text{runtime}_1 \right).$$

For example, for L_{10} we have $0.31 \cdot \text{runtime}_1$, while for L_{22} we obtain $0.35 \cdot \text{runtime}_1$. In particular, when p increases, the constant γ decreases slightly (see Table 3).

Table 3: The row corresponding to L presents the best choice of L . In particular, we have $L_{10} = [3, 7]$ and $L_{22} = [3, 5, 11]$. The row corresponding with max. γ determines the constant factor in the running time of SuperSolver (i.e., Eq. (5)), while avg. γ presents the average running time of SuperSolver.

$\log_p$	32	64	96	128	192	256	384	512
max. γ	1.64	0.91	0.67	0.53	0.38	0.31	0.24	0.20
avg. γ	0.51	0.28	0.21	0.16	0.13	0.11	0.08	0.07
L	L_{10}	L_{10}	L_{10}	L_{10}	L_{22}	L_{22}	L_{22}	L_{22}

4 Subfield Curve Searching Over Specific Fields

This section describes an optimized Depth-First-Search (DFS) traversal of the 2-isogeny trees for finding subfield curves. The algorithm takes inspiration from the DFS technique of [ACC⁺19].

Let us focus on supersingular elliptic curves defined over $\mathbb{F}_{p^2}$ with $(p+1)^2$ points, which implicitly implies the existence of large 2-torsion points on the curve. Fix a height parameter $h \leq a$, and let $\{P, Q\}$ be a basis of the 2^h -torsion subgroup of a given supersingular elliptic curve $\mathcal{E}: y^2 = x^3 + Ax^2 + x$. Next, we inspect the 2-isogeny subtree with root $\mathcal{E}$ and 2^h leaves, excluding the 2-isogeny with kernel generated by $[2^{h-1}]Q = (0, 0)$, as described in Figure 3.

The key observation of the above setting is that it allows us to compute 2-isogenies through kernel points, and thus, we work over the curves instead of the j-invariants. Therefore, we can avoid calculating square roots when inspecting the 2-isogeny tree rooted at $\mathcal{E}$. We could think that we require a field inversion to compute the j-invariant of each curve in Figure 3, but we highlight that we only need the numerator $(a + ib)$ and the denominator $(c + id)$ of each j-invariant and check if $ad = bc$ holds. That is, we can avoid the field inversions in Figure 3.

Searcher ₂ ($\mathcal{E}, P, Q, h$)	
1 :	Initialize an empty stack for storage of nodes, S
2 :	Push the root $\mathbf{z}_\emptyset = \{\mathcal{E}, \emptyset, P, Q\}$ onto S
3 :	Pop the first element in S , $\mathbf{z}_b = \{\mathcal{E}_b, \mathbf{b}, P_b, Q_b\}$ where $\mathbf{b} \in \{0, 1\}^n$
4 :	if $j(\mathcal{E}_b) \in \mathbb{F}_p$
5 :	return $\mathbf{b}$
6 :	endif
7 :	if $n < (h - 1)$
8 :	Calculate the 2-isogeny $\phi_{b0}: \mathcal{E}_b \rightarrow \mathcal{E}_{b0}$ with $\ker \phi_{b0} = \langle [2^{h-n-1}]P_b \rangle$
9 :	Compute $P_{b0} = \phi_{b0}(P_b)$ and $Q_{b0} = \phi_{b0}([2]Q_b)$
10 :	Calculate the 2-isogeny $\phi_{b1}: \mathcal{E}_b \rightarrow \mathcal{E}_{b1}$ with $\ker \phi_{b0} = \langle [2^{h-n-1}](P_b + Q_b) \rangle$
11 :	Compute $P_{b1} = \phi_{b1}(P_b + Q_b)$ and $Q_{b1} = \phi_{b1}([2]Q_b)$
12 :	Push $\mathbf{z}_{b1} = \{\mathcal{E}_{b1}, \mathbf{b1}, P_{b1}, Q_{b1}\}$ onto S
13 :	Push $\mathbf{z}_{b0} = \{\mathcal{E}_{b0}, \mathbf{b0}, P_{b0}, Q_{b0}\}$ onto S
14 :	Go to Line 3
15 :	endif
16 :	if $n = (h - 1)$
17 :	Calculate the 2-isogeny $\phi_{b0}: \mathcal{E}_b \rightarrow \mathcal{E}_{b0}$ with kernel $\ker \phi_{b0} = \langle P_b \rangle$
18 :	Calculate the 2-isogeny $\phi_{b1}: \mathcal{E}_b \rightarrow \mathcal{E}_{b1}$ with kernel $\ker \phi_{b1} = \langle P_b + Q_b \rangle$
19 :	Push $\mathbf{z}_{b1} = \{\mathcal{E}_{b1}, \mathbf{b1}, \mathbf{0}_{\mathcal{E}_{b1}}, \mathbf{0}_{\mathcal{E}_{b1}}\}$ onto S
20 :	Push $\mathbf{z}_{b0} = \{\mathcal{E}_{b0}, \mathbf{b0}, \mathbf{0}_{\mathcal{E}_{b0}}, \mathbf{0}_{\mathcal{E}_{b0}}\}$ onto S
21 :	Go to Line 3
22 :	endif
23 :	if S is empty
24 :	return $\perp$
25 :	else
26 :	Go to Line 3
27 :	endif

Figure 3: Description of our subfield curve search algorithm, **Searcher₂**. The input must satisfy $\mathcal{E}$ is a supersingular elliptic curve defined over $\mathbb{F}_{p^2}$, h is a positive integer, and $\mathcal{E}[2^h] = \langle P, Q \rangle$. The integer n determines the length of the bitstring $\mathbf{b}$. For the empty bitstring $\mathbf{b} = \emptyset$, we set $n = 0$. The value of $\mathbf{b}$ describes the 2-isogeny path of length n going from $\mathcal{E} = \mathcal{E}_\emptyset$ to $\mathcal{E}_n$. For each $i = 0, 1$, the node $\mathbf{z}_{bi} = \{\mathcal{E}_{bi}, \mathbf{bi}, P_{bi}, Q_{bi}\}$ satisfies $\mathcal{E}_{bi}[2^{h-n-1}] = \langle P_{bi}, Q_{bi} \rangle$ and $\ker \hat{\phi}_{bi} = \langle [2^{h-n-2}]Q_{bi} \rangle$.

We state in Proposition 1 the time complexity of $\mathcal{O}(p^{1/2})$ operations for our subfield curve search procedure presented in Figure 3.

Proposition 1. *Let $p = 2^a \cdot f - 1$ be a prime with $a \geq \frac{1}{2} \log_2 p$, and let $h = \log_2 \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right)$. The algorithm in Figure 3 has a running time of $\mathcal{O}(2^h)$ field operations over $\mathbb{F}_{p^2}$, and it has a memory complexity polynomial in $\log_2 p$ and h .*

Proof. Let us focus on the worst-case scenario (i.e., we inspect all the nodes in the 2-isogeny subtree). Notice that computing the kernel point generators of ϕ_{b0} and ϕ_{b1} can be achieved at the cost of $2 \cdot (h - n - 1)$ point doublings, while the required points in Lines 9 and 11 only need one extra point addition and doubling. Additionally, we need to push four points through 2-isogenies and two 2-isogeny codomain calculations. Thus, inspecting the 2-isogeny subtree requires

- $\sum_{k=0}^h 2^k = 2^{h+1} - 1 \approx 2 \cdot 2^h$ subfield checks,
- $\sum_{k=0}^{h-2} (2(h-k-1) + 1) \cdot 2^k = 5 \cdot 2^{h-1} - 2h - 3 \approx \frac{5}{2} \cdot 2^h$ point doublings,
- $\sum_{k=0}^{h-1} 2^k = 2^h - 1 \approx 2^h$ point additions,
- $\sum_{k=0}^{h-2} 4 \cdot 2^k = 3 \cdot 2^h - 4 \approx 2 \cdot 2^h$ isogeny evaluations, and
- $\sum_{k=0}^{h-1} 2 \cdot 2^k = 2^{h+1} - 2 \approx 2 \cdot 2^h$ isogeny codomain calculations.

In particular, we get an asymptotical running time of $\mathcal{O}(2^h)$ field operations since each point and isogeny operation takes $\mathcal{O}(1)$ field operations. The memory storage is limited to $(2h+1)$ stack elements, which gives a memory complexity polynomial in $\log_2 p$ and h . $\square$

Explicit cost on the Kummer line. Given the lesser cost of adding x -only points on the Kummer line than on the elliptic curve, we detailed the running of the proposed method in terms of field operation through x -only points. Thus,

1. The points in $\mathbf{z}_b$ represent projective x -only points of the form $(X : Z)$.
2. The node $\mathbf{z}_b$ has an extra point, $\mathbf{z}_b = \{\mathcal{E}_b, b, x(P_b), x(Q_b), x(R_b)\}$ where $R_b = P_b - Q_b$.
3. The elliptic curve in $\mathbf{z}_b$ is represented by its projective representation, $(A' + 2C : 4C)$ where $A = A'/C$.
4. The additional point $x(R_b)$ implies an extra 2-isogeny evaluation per isogeny, $x(R_{b0}) = x(\phi_{b0}(P_b - [2]Q_b))$ and $x(R_{b1}) = x(\phi_{b1}(P_b - Q_b))$.

We summarize in Lemma 1 the explicit time complexity of the algorithm `Searcher2` instantiated over the Kummer line.

Lemma 1. *Let $p = 2^a \cdot f - 1$ be a prime with $a \geq \frac{1}{2} \log_2 p$, and let $h = \log_2 \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right)$. The algorithm in Figure 3 instantiated over the Kummer line has a time complexity bounded by*

$$132 \cdot \frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \text{ mul.}$$

Under Assumption 1, algorithm in Figure 3 is likely to succeed, and on average a subfield curve is found with around

$$33 \cdot \frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \text{ mul.}$$

Proof. Notice that Assumption 1 implies the algorithm in Figure 3 is likely to find a subfield curve. Next, let us focus on the worst-case scenario (i.e., we inspect all the nodes in the 2-isogeny subtree, and thus the algorithm returns $\perp$). This time, inspecting the 2-isogeny subtree performs

- $\sum_{k=0}^h 2^k = 2^{h+1} - 1 \approx 2 \cdot 2^h$ subfield checks,
- $\sum_{k=0}^{h-2} (2(h-k-1) + 1) \cdot 2^k = 5 \cdot 2^{h-1} - 2h - 3 \approx \frac{5}{2} \cdot 2^h$ point doublings,
- $\sum_{k=0}^{h-2} 2 \cdot 2^k = 2^h - 2 \approx 2^h$ point additions,
- $\sum_{k=0}^{h-2} 6 \cdot 2^k = 3 \cdot 2^h - 6 \approx 3 \cdot 2^h$ isogeny evaluations, and
- $\sum_{k=0}^{h-1} 2 \cdot 2^k = 2^{h+1} - 2 \approx 2 \cdot 2^h$ isogeny codomain calculations.

According to Table 2, and since one can perform a subfield check with $2M + 4S + 16A + 2\text{mul}$ operations,⁵ we get a running time bounded by

$$\text{runtime}_2 = (30M + 19S + 4\text{mul})2^h = 132 \cdot \frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \text{mul}.$$

Now, observe that the algorithm Searcher_2 inspects $2^{h+1} - 1$ curves, doubling the required curves $2^h = \frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p}$. Therefore, we can restrict the search to the condition

$$h = \log_2 \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right) - 1.$$

Consequently, the runtime of the algorithm Searcher_2 is bounded by

$$\text{runtime}_2 = 132 \cdot 2^h \text{mul} = \frac{132}{2} \cdot \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right) \text{mul} = 66 \cdot \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right) \text{mul}. \quad (6)$$

In particular, following Remark 1, the expected *average* running time of Searcher_2 becomes

$$\frac{1}{2} \cdot \text{runtime}_2 = 33 \cdot \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right) \text{mul}.$$

□

4.1 The Impact of Using 4-isogenies instead of 2-isogenies

Computing 4-isogenies is much cheaper than composing two consecutive 2-isogenies (this is already one optimization trick used in many isogeny cryptographic implementations).⁶ However, a few differences exist when employing 4-isogenies instead of 2-isogenies in the subfield curve search algorithm. To highlight the usage of 4-isogenies in Searcher_2 , we denote the 4-variant algorithm by Searcher_4 .

Setup when employing 4-isogenies. Following the notation from Fig. 3, the input must satisfy $\mathcal{E}[4^h] = \langle P, Q \rangle$. In addition, the integer n determines the length of the string $\mathbf{b} \in \{0, 1, 2, 3\}^n$, and we denote $\mathbf{z}_\mathbf{b} = \{\mathcal{E}, \mathbf{b}, x(P_\mathbf{b}), x(Q_\mathbf{b}), x(R_\mathbf{b})\}$ where $R_\mathbf{b} = P_\mathbf{b} - Q_\mathbf{b}$.

Calculating Codomain 4-isogenies when $n < (h - 1)$. We need to calculate four different 4-isogenies:

$$\begin{aligned} \phi_{\mathbf{b}0}: \mathcal{E}_\mathbf{b} &\rightarrow \mathcal{E}_{\mathbf{b}0} \text{ with } \ker \phi_{\mathbf{b}0} = \langle [4^{h-n-1}](P_\mathbf{b}) \rangle, \\ \phi_{\mathbf{b}1}: \mathcal{E}_\mathbf{b} &\rightarrow \mathcal{E}_{\mathbf{b}1} \text{ with } \ker \phi_{\mathbf{b}1} = \langle [4^{h-n-1}](P_\mathbf{b} + Q_\mathbf{b}) \rangle, \\ \phi_{\mathbf{b}2}: \mathcal{E}_\mathbf{b} &\rightarrow \mathcal{E}_{\mathbf{b}2} \text{ with } \ker \phi_{\mathbf{b}2} = \langle [4^{h-n-1}](P_\mathbf{b} + [2]Q_\mathbf{b}) \rangle, \text{ and} \\ \phi_{\mathbf{b}3}: \mathcal{E}_\mathbf{b} &\rightarrow \mathcal{E}_{\mathbf{b}3} \text{ with } \ker \phi_{\mathbf{b}3} = \langle [4^{h-n-1}](P_\mathbf{b} + [3]Q_\mathbf{b}) \rangle. \end{aligned}$$

Regarding operations in the Kummer line, the cost of computing the above four isogenies includes $3(h - n - 1)$ x -only point quadrupling and three x -only point additions.

⁵Computing the numerator and denominator of $j(\mathcal{E}) = (a + ib)/(c + id)$ from $(A' + 2C : 4C)$ requires $2M + 4S + 16A$, and checking $j(\mathcal{E}) \in \mathbb{F}_p$ reduce to verifying $ad = bc$.

⁶We exclude the analysis for employing 8-isogenies since there are no known efficient 8-isogeny formulas better than performing a composition of three 2-isogenies or one 2-isogeny together with a single 4-isogeny.

Calculating Codomain 4-isogenies when $n = (h - 1)$. We need to calculate four different 4-isogenies:

$$\begin{aligned}\phi_{b0}: \mathcal{E}_b &\rightarrow \mathcal{E}_{b0} \text{ with } \ker \phi_{b0} = \langle P_b \rangle, \\ \phi_{b1}: \mathcal{E}_b &\rightarrow \mathcal{E}_{b1} \text{ with } \ker \phi_{b1} = \langle P_b + Q_b \rangle, \\ \phi_{b2}: \mathcal{E}_b &\rightarrow \mathcal{E}_{b2} \text{ with } \ker \phi_{b2} = \langle P_b + [2]Q_b \rangle, \text{ and} \\ \phi_{b3}: \mathcal{E}_b &\rightarrow \mathcal{E}_{b3} \text{ with } \ker \phi_{b3} = \langle P_b + [3]Q_b \rangle = \langle P_b - Q_b \rangle.\end{aligned}$$

Observe that the above kernel calculations add two extra x -only point additions.

Pushing Points Through 4-isogenies. We need to push a total of twelve x -only points through the isogenies:

$$\begin{aligned}x(P_{b0}) &= x(\phi_{b0}(P_b)), & x(Q_{b0}) &= x(\phi_{b0}([4]Q_b)), & x(R_{b0}) &= x(\phi_{b0}(P_b - [4]Q_b)) \\ x(P_{b1}) &= x(\phi_{b1}(P_b + Q_b)), & x(Q_{b1}) &= x(\phi_{b1}([4]Q_b)), & x(R_{b1}) &= x(\phi_{b1}(P_b - [3]Q_b)) \\ x(P_{b2}) &= x(\phi_{b2}(P_b + [2]Q_b)), & x(Q_{b2}) &= x(\phi_{b2}([4]Q_b)), & x(R_{b2}) &= x(\phi_{b2}(P_b - [2]Q_b)) \\ x(P_{b3}) &= x(\phi_{b3}(P_b + [3]Q_b)), & x(Q_{b3}) &= x(\phi_{b3}([4]Q_b)), & x(R_{b3}) &= x(\phi_{b3}(P_b - Q_b)).\end{aligned}$$

Notice that the above points to be evaluated imply six extra x -only point additions for computing $x(P_{b0})$, $x(P_{b1})$, $x(P_{b2})$, $x(P_{b3})$, $x(R_{b0})$, $x(R_{b1})$, $x(R_{b2})$, and $x(R_{b3})$.

We formalize the running time of `Searcher4` through Lemma 2, which relies on the following 4-variant version of Assumption 1.

Lemma 2. *Let $p = 4^a \cdot f - 1$ be a prime with $a \geq \frac{1}{2} \log_4 p$, and let $h = \log_4 \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right)$. The 4-isogeny variant of the algorithm in Figure 3 instantiated over the Kummer line has a time complexity bounded by*

$$82 \cdot \frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \text{ mul.}$$

Under Assumption 1, the 4-isogeny variant of the algorithm in Figure 3 is likely to succeed, and on average a subfield curve is found with around

$$30.75 \cdot \frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \text{ mul.}$$

Proof. Notice that Assumption 1 implies the algorithm `Searcher4` is likely to find a subfield curve. Next, let us focus on the worst-case scenario (i.e., we inspect all the nodes in the 4-isogeny subtree, and thus the algorithm return $\perp$). This time, inspecting the 4-isogeny subtree performs

$$\begin{aligned}- \sum_{k=0}^h 4^k &= \frac{1}{3} \cdot (4^{h+1} - 1) \approx \frac{4}{3} \cdot 4^h \text{ subfield checks,} \\ - \sum_{k=0}^{h-2} (6(h-k-1) + 1) \cdot 4^k &= \frac{2}{3} \cdot (4^h - 3h - 1) \approx \frac{2}{3} \cdot 4^h \text{ point doublings,} \\ - 2 \cdot 4^{n-1} + \sum_{k=0}^{h-2} 8 \cdot 4^k &= \frac{2}{3} \cdot 4^h + \frac{2}{3} (4^h - 4) \approx \frac{4}{3} \cdot 4^h \text{ point additions,} \\ - \sum_{k=0}^{h-2} 12 \cdot 4^k &= 4^h - 4 \approx 3 \cdot 4^h \text{ isogeny evaluations, and} \\ - \sum_{k=0}^{h-1} 4 \cdot 4^k &= \frac{4}{3} \cdot (4^h - 1) \approx \frac{4}{3} \cdot 4^h \text{ isogeny codomain calculations.}\end{aligned}$$

Then, similarly to Lemma 1, we obtain a running time bounded by

$$\text{runtime}_4 = \frac{974}{12} \cdot 4^h \text{ mul} \approx 82 \cdot 4^d \text{ mul} = 82 \cdot \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right) \text{ mul.} \quad (7)$$

However, differently from the algorithm `Searcher2`, observe that the algorithm `Searcher4` inspects $\frac{1}{3} \cdot (4^{h+1} - 1)$ curves. If we restrict the search to the condition

$$h = \log_4 \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right) - 1$$

then we end up by inspecting only $\frac{1}{3} \cdot (4^h - 1)$ curves, which is less than $\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p}$. Therefore, we set

$$h = \log_4 \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right)$$

for `Searcher4`, which implies inspecting $\frac{4}{3} \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right)$ curves. Hence, from Remark 1, the expected *average* running time of `Searcher4` turns out to be

$$\frac{1}{2} \cdot \left(\frac{3}{4} \cdot \text{runtime}_4 \right) = 30.75 \cdot \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right) \text{ mul}$$

□

Remark 2. We note that Lemma 2 does not include the inspection of the intermediate 2-isogeny nodes on the 4-isogeny tree. We discuss the impact of including this extra check in Appendix A.1.

4.2 Applicability to primes of the form $p = d^a \cdot f - 1$

It is worth noting that the algorithm `Searcher2` can be easily extended to any d -isogeny, with the key difference that each node in the tree does not have two children but d children. For simplicity, we label the d -variant algorithm by `Searcherd`. If d is a small integer number, then `Searcherd` has a cost of $\mathcal{O}(d^h)$ field multiplications.

Setup when employing d -isogenies. Following the notation from Fig. 3, the input must satisfy $\mathcal{E}[d^h] = \langle P, Q \rangle$. In addition, the integer n determines the length of the string $\mathbf{b} \in \{0, \dots, d-1\}^n$, and we denote $\mathbf{z}_\mathbf{b} = \{\mathcal{E}, \mathbf{b}, \mathbf{x}(P_\mathbf{b}), \mathbf{x}(Q_\mathbf{b}), \mathbf{x}(R_\mathbf{b})\}$ where $R_\mathbf{b} = P_\mathbf{b} - Q_\mathbf{b}$.

Calculating Codomain d -isogenies when $n < (h-1)$. We need to calculate d different d -isogenies:

$$\phi_{\mathbf{b}j} : \mathcal{E}_\mathbf{b} \rightarrow \mathcal{E}_{\mathbf{b}j} \text{ with } \ker \phi_{\mathbf{b}j} = \langle [d^{h-n-1}](P_\mathbf{b} + [j]Q_\mathbf{b}) \rangle \text{ for each } j := 0, \dots, d-1.$$

This time, the cost of computing the above d isogenies includes $3(h-n-1)$ x -only point multiplication by d and $(d-1)$ x -only point additions.

Calculating Codomain d -isogenies when $n = (h-1)$. We need to calculate d different d -isogenies:

$$\phi_{\mathbf{b}j} : \mathcal{E}_\mathbf{b} \rightarrow \mathcal{E}_{\mathbf{b}j} \text{ with } \ker \phi_{\mathbf{b}j} = \langle P_\mathbf{b} + [j]Q_\mathbf{b} \rangle \text{ for each } j := 0, \dots, d-1.$$

Similarly, the cost of computing the above d isogenies only includes $(d-2)$ x -only point additions.

Pushing Points Through d -isogenies. We need to push a total of $3d$ x -only points through the isogenies:

$$\begin{aligned} \mathbf{x}(P_{\mathbf{b}j}) &= \mathbf{x}(\phi_{\mathbf{b}j}(P_\mathbf{b} + [j]Q_\mathbf{b})), & \mathbf{x}(Q_{\mathbf{b}j}) &= \mathbf{x}(\phi_{\mathbf{b}j}([d]Q_\mathbf{b})), \text{ and} \\ \mathbf{x}(R_{\mathbf{b}j}) &= \mathbf{x}(\phi_{\mathbf{b}j}(P_\mathbf{b} - [d-j]Q_\mathbf{b})) \end{aligned}$$

for each $j := 0, \dots, d-1$. Those calculations includes $2(d-1)$ x -only point additions.

On the running time of Searcher_{12} . To illustrate the impact of the algorithm Searcher_d , let us focus on primes of the form $p = (4 \cdot 3)^a \cdot f - 1$, with $f \in \mathbb{N}$ and $a \geq \frac{1}{2} \log_{12} p$ (e.g., this kind of primes are used in [CSCD⁺23, DLRW24]). Hence, let us take $d = 12$ and consider the following.⁷

- A point multiplication by twelve requires calculating two-point doubling and one-point tripling. This gives a cost of $15M + 9S$ field operations.
- A 12-isogeny can be efficiently decomposed as the composition of a 4-isogeny followed by a 3-isogeny:
 - Computing a 12-isogeny curve codomain includes one point tripling, one 4-isogeny curve codomain calculation, one 4-isogeny evaluation, and one last 3-isogeny curve codomain calculation. This gives a cost of $15M + 14S$ field operations.
 - Pushing a point through a 12-isogeny implies pushing the point through a 4-isogeny and a 3-isogeny. This gives a cost of $10M + 4S$ field operations.

We formalize the running time of Searcher_{12} through Lemma 3, which relies on the following 12-variant version of Assumption 1.

Lemma 3. *Let $p = 12^a \cdot f - 1$ be a prime with $a \geq \frac{1}{2} \log_{12} p$, and let $h = \log_{12} \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right)$. The 12-isogeny variant of the algorithm in Figure 3 instantiated over the Kummer line has a time complexity bounded by*

$$127 \cdot \frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \text{ mul.}$$

Under Assumption 1, the 12-isogeny variant of the algorithm in Figure 3 is likely to succeed, and on average a subfield curve is found with around

$$58.21 \cdot \frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \text{ mul.}$$

Proof. Notice that Assumption 1 implies the algorithm Searcher_{12} is likely to find a subfield curve. Next, let us focus on the worst-case scenario (i.e., we inspect all the nodes in the 12-isogeny subtree, and thus the algorithm return $\perp$). This time, inspecting the 12-isogeny subtree performs

- $\sum_{k=0}^h 12^k = \frac{1}{11} \cdot (12^{h+1} - 1) \approx \frac{12}{11} \cdot 12^h$ subfield checks,
- $\sum_{k=0}^{h-2} 3(h-k-1) \cdot 12^k = \frac{3}{121} \cdot (12^h - 11h - 1) \approx \frac{3}{121} \cdot 12^h$ point multiplications by twelve,
- $10 \cdot 12^{h-1} + \sum_{k=0}^{h-2} 33 \cdot 12^k = \frac{10}{12} \cdot 12^h + \frac{1}{4}(12^h - 12) \approx \frac{13}{12} \cdot 12^h$ point additions,
- $\sum_{k=0}^{h-2} 36 \cdot 12^k = \frac{3}{11} \cdot (12^h - 12) \approx \frac{3}{11} \cdot 12^h$ isogeny evaluations, and
- $\sum_{k=0}^{h-1} 12 \cdot 12^k = \frac{12}{11} \cdot (12^h - 1) \approx \frac{12}{11} \cdot 12^h$ isogeny codomain calculations.

Then, running time of the algorithm Searcher_{12} is bounded by

$$\text{runtime}_{12} = \frac{183460}{1452} \cdot 12^h \text{ mul} \approx 127 \cdot 12^h \text{ mul} = 127 \cdot \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right) \text{ mul.} \quad (8)$$

⁷We focus on 12-isogenies instead of 6-isogenies since we experimentally observed about the same complexities. In particular, we consider that for primes of the form $p = 2^a \cdot 3^b \cdot f - 1$ we have $a \approx b \log_2 3 \approx 1.58b$, and using $d = 12$ allows us to inspect a (slightly) bigger number of curves than when employing $d = 6$, at a negligible impact on the running time.

Furthermore, the *expected* average running time of `Searcher12` becomes

$$\frac{1}{2} \left(\frac{11}{12} \cdot \text{runtime}_{12} \right) \approx 58.21 \cdot \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right) \text{ mul.}$$

□

4.3 Adapting the Fast Subfield Root Detection in $\mathcal{X}(\overline{\mathbb{F}}_p, \ell)$

Our description of algorithm `Searcherd` explicitly works over the elliptic curves, which would require the calculation of $j(\mathcal{E}_b)$ (adding a field inversion calculation) if using algorithm `NeighbourInFp`. For instance, from the optimized instantiation of algorithm `Searcherd` over the Kummer line (see Section 4), we can calculate the numerator u and denominator v of each j -invariant $j = u \cdot v^{-1}$. Then, we cannot directly employ algorithm `NeighbourInFp` from [SCS22] without the calculation of v^{-1} .⁸ Including that field inversion would increase the overall complexity of our searcher algorithm by a logarithm factor $\log_2 p$, therefore increasing the runtime to $\tilde{O}(p^{1/2})$.

However, to avoid the calculation of the inverse of v , we next adapt the procedure `NeighbourInFp` to our case study. Let us assume we have $u, v \in \mathbb{F}_{p^2}$ such that $v \neq 0$ and $j(\mathcal{E}) = \frac{u}{v}$ is the j -invariant of a supersingular elliptic curve $\mathcal{E}$, and consider the polynomial

$$\Delta_\ell(x, u, v) = v^{d_\ell} \cdot \Phi_\ell \left(x, \frac{u}{v} \right).$$

Notice the coefficients of $\Delta_\ell(x, u, v)$ do not require the explicit value of v^{-1} , and we have

$$\begin{aligned} g_1 &= \frac{1}{2} (\Phi_\ell + \pi(\Phi_\ell)) = \frac{1}{2w} (\pi(v^{d_\ell}) \cdot \Delta_\ell + v^{d_\ell} \cdot \pi(\Delta_\ell)), \text{ and} \\ g_2 &= -\frac{i}{2} (\Phi_\ell - \pi(\Phi_\ell)) = -\frac{i}{2w} (\pi(v^{d_\ell}) \cdot \Delta_\ell - v^{d_\ell} \cdot \pi(\Delta_\ell)), \end{aligned}$$

where $w = (v \cdot \pi(v))^{d_\ell}$ and $\pi: x \mapsto x^p$ denotes the Frobenius map. Re-writing g_1 and g_2 in terms of $\Theta_\ell = \pi(v)^{d_\ell} \cdot \Delta_\ell$ gives

$$g_1 = \frac{1}{2w} (\Theta_\ell + \pi(\Theta_\ell)) \quad \text{and} \quad g_2 = -\frac{i}{2w} (\Theta_\ell - \pi(\Theta_\ell)).$$

Since $\gcd(g_1, g_2) = \gcd(w \cdot g_1, w \cdot g_2)$ holds, we can evaluate Θ_ℓ at (u, v) as follows:

- Calculate $u^2, \dots, u^{d_\ell}, v^2, \dots, v^{d_\ell}$, and $t = \pi(v)^{d_\ell} = \pi(v^{d_\ell})$.
- Compute $z_m = t \cdot u^n \cdot v^m$ for each $n, m \in \{0 \dots d_\ell\}$ such that $d_\ell = n + m$.
- Multiply each coefficient z_m by the corresponding coefficients of Φ_ℓ .

The above third step essentially costs what `EvaluateModularPolynomial` does for calculating $\Phi_\ell(x, j)$ but this time multiplying by z_m instead of j^m for obtaining $\Theta_\ell(x, u, v)$. Then, we can evaluate Θ_ℓ at (u, v) by performing $(9d_\ell(d_\ell - 1) + 9(d_\ell - 1))\text{mul}$ operations. On the other hand, `InverseFreeGCD`($w \cdot g_1, w \cdot g_2$) still takes $(2d_\ell + 1)(d_\ell + 1) - 6$ multiplications in $\mathbb{F}_p$.⁹ Thus, our modified subfield root detection algorithm `ScaledNeighbourInFp`(ℓ, j) requires at most $(11d_\ell^2 + 3d_\ell - 14)\text{mul}$ operations. We give a high-level description of the algorithm `ScaledNeighbourInFp` in Figure 4.

⁸Corte-Real Santos, Costello, and Shi highlight that having the j -invariants explicitly computed is critical in their subfield root detection algorithm [SCS22, Remark 2].

⁹This is because $\Theta_\ell(x, u, v) = w \cdot \Phi_\ell(x, \frac{u}{v})$ with $w \in \mathbb{F}_p$ hold, and its leading coefficient is w itself.

ScaledNeighbourInFp(ℓ, u, v)	
1 :	$f^* \leftarrow \text{EvaluateScaledModularPolynomial}(\ell, u, v)$
2 :	Compute $g_1^* = \frac{1}{2}(f^* + \pi(f^*))$
3 :	Compute $g_2^* = -\frac{i}{2}(f^* - \pi(f^*))$
4 :	return $\text{InverseFreeGCD}(g_1^*, g_2^*)$

Figure 4: Description of our modified subfield root detection algorithm. The input must satisfy that $j(\mathcal{E}) = \frac{u}{v}$ for some supersingular curve $\mathcal{E}$ defined over $\mathbb{F}_{p^2}$. The subroutine $\text{EvaluateScaledModularPolynomial}(\ell, j)$ evaluates $\Theta_\ell(x, y)$ at (u, v) as described above.

4.3.1 Optimized Subfield Curve Search Procedure.

Given a list of t small odd primes $L = [\ell_1, \ell_2, \dots, \ell_t]$, we suggest modifying the DFS algorithm in Figure 3 by including the subfield curve inspection for each $\ell \in L$ in Line 4. In that way, we increase the number of elliptic curves inspected, i.e., it allows us to detect if there is one ℓ -isogenous subfield curve among the $(\ell + 1)$ possibilities. That optimization helps to increase the number of inspected curves in Searcher_d by a factor of $(1 + \sum_{\ell \in L} d_\ell)$ times bigger. Therefore, we can decrease the value of h to (at most)

$$h = \log_d \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right) - \left\lceil \log_d \left(1 + \sum_{\ell \in L} d_\ell \right) \right\rceil$$

and still inspect around $\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p}$ curves.

4.3.2 Optimizing the choice of each ℓ_k

The situation here differs from [SCS22] because we do not need to calculate square roots to compute 2-isogenous curves. However, we can isolate the cost associated with inspecting through d -isogenies in algorithm Searcher_d , then split the total associated cost to $\text{ScaledNeighbourInFp}$, and maximize the number of inspected curves as much as possible.

In what follows, we label the modified algorithm, which includes the subfield curve search, by SuperSearcher_d to distinguish it from the baseline algorithm Searcher_d . We adopt the subfield cost for each algorithm step as in [SCS22] to better understand the difference in cost of inspecting $\mathcal{X}(\overline{\mathbb{F}}_p, \ell)$ when employing NeighbourInFp and $\text{ScaledNeighbourInFp}$; more precisely, we take that cost as

$$\text{cost}_\ell^* = \frac{1}{d_\ell} (\text{cost of ScaledNeighbourInFp}) \leq \frac{1}{d_\ell} (11d_\ell^2 + 3d_\ell - 14) \text{ mul.}$$

Table 4 presents and compares the concrete cost for inspecting $\mathcal{X}(\overline{\mathbb{F}}_p, \ell)$ when employing NeighbourInFp and $\text{ScaledNeighbourInFp}$. Observe that the procedure $\text{ScaledNeighbourInFp}$ is slightly costlier than NeighbourInFp , but we highlight that the algorithm Searcher_d with NeighbourInFp adds logarithmic factors in the overall running time while with $\text{ScaledNeighbourInFp}$ does not. We thus consider $(c, d) \in \{(132, 2), (82, 4), (127, 12)\}$ and the running time of algorithm SuperSearcher_d to be bounded by

$$\text{runtime}_d^* = \underbrace{\frac{1}{d^{\left\lceil \log_d \left(1 + \sum_{\ell \in L} d_\ell \right) \right\rceil}} \left(c + \frac{d}{d-1} \cdot \sum_{\ell \in L} \overbrace{d_\ell \cdot \text{cost}_\ell^*}^{\text{ScaledNeighbourInFp}} \right)}_{\gamma(L)} \cdot d^h \text{ mul,} \quad (9)$$

Table 4: The row corresponding to cost_ℓ presents the values from [SCS22, Table 3].

ℓ :	3	5	7	11	13	17	19	23	29	31	37	41	43
d_ℓ :	4	6	8	12	14	18	20	24	30	32	38	42	44
cost_ℓ :	16.3	24.5	32.6	48.8	56.8	72.8	80.9	96.9	120.9	128.9	152.9	168.9	176.9
cost_ℓ^* :	25.8	35.8	43.9	61.4	69.2	85.9	93.8	110.2	134.4	142.2	166.3	182.5	190.4

Observe that $\gamma(L)$ does not depend on neither $\log_2 p$ nor d ; therefore, we can consider it a constant factor. With that in mind, we want to minimize the value of $\gamma(L) \leq c$. We combine the concrete costs from Table 4 to select the best candidate of L with at most thirteen elements (any other larger prime number ℓ implies a higher number of field operations when inspecting $\mathcal{X}(\mathbb{F}_p, \ell)$). More precisely, we perform a brute-force search on $\mathbf{b} \in \{0, 1\}^{13}$ to determine

$$\underset{\mathbf{b}}{\text{argmin}} \gamma(L_{\mathbf{b}}),$$

where $L_{\mathbf{b}}$ denotes the list having the i^{th} prime ℓ_i if the i^{th} bit of $\mathbf{b}$ is nonzero.

Remark 3. Given that algorithm SuperSearcher_d inspects

$$N = \frac{d}{d-1} \cdot d^h \cdot (1 + \sum_{\ell \in L} d_\ell)$$

curves, where $h = \log_d \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right) - \lfloor \log_d (1 + \sum_{\ell \in L} d_\ell) \rfloor$, we consider that if N is smaller than $\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p}$, then algorithm SuperSearcher_d needs to increase the value of h by one to inspect at least $\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p}$ curves; in other words: if $N < \frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p}$ then the value of γ needs to be multiplied by d .

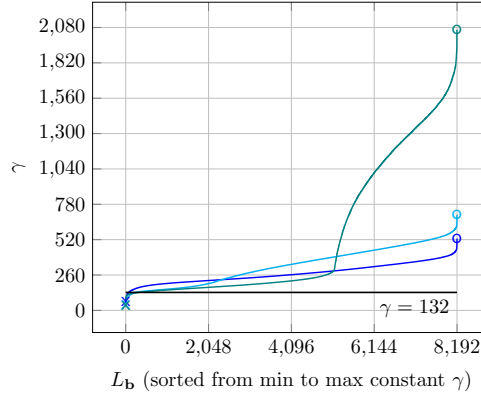


Figure 5: Value of $\gamma(L_{\mathbf{b}})$ for each $\mathbf{b} \in \{0, 1\}^{13}$. The x-axis represents the choice of the list $L_{\mathbf{b}}$ sorted from the one with minimal constant $\min \gamma(L_{\mathbf{b}})$ to the one with maximal constant $\max \gamma(L_{\mathbf{b}})$. The y-axis determines the constant $\gamma(L_{\mathbf{b}})$ as given in Equation (9). We mark the maximum $\max \gamma(L_{\mathbf{b}})$ by $\circ$, and the minimum $\min \gamma(L_{\mathbf{b}})$ with $\times$. The curve in blue ink color corresponds with algorithm SuperSearcher , while the one with cyan (resp. teal) ink presents the values for algorithm SuperSearcher_4 (resp. $\text{SuperSearcher}_{12}$). The big jumps on the graph concerning $\text{SuperSearcher}_{12}$ come from the values of γ being multiplied by $d = 12$ as stated in Remark 3.

We plot all the values of $\gamma(L_{\mathbf{b}})$ for each $\mathbf{b} \in \{0, 1\}^{13}$ in Figure 5. In particular, we obtain that the best choice for L is

- $L_{10} = [3, 7]$ with $\gamma(L_{10}) = 65$ when performing the algorithm SuperSearcher_2 ,
- $L_{10} = [3, 7]$ with $\gamma(L_{10}) = 42.96$ when using the algorithm SuperSearcher_4 ,
- $L_6 = [3, 5]$ with $\gamma(L_6) = 39.49$ when employing the algorithm $\text{SuperSearcher}_{12}$.

We present the expected *average* running time of SuperSearcher_d in Corollary 1.

Corollary 1. *Let $(d, L) \in \{(2, L_{10}), (4, L_{10}), (12, L_6)\}$, and let $p = d^a \cdot f - 1$ be a prime number for some $a, f \in \mathbb{Z}^+$. Consider $(\gamma, d) \in \{(20, 2), (19.83, 4), (19.75, 12)\}$. Under Assumption 1, the algorithm SuperSearcher_d is likely to find a subfield curve with an expected average running time equal to*

$$\frac{1}{2} \cdot \left(\frac{(d-1) \cdot d^{\lceil \log_d (1 + \sum_{\ell \in L} d_\ell) \rceil}}{d \cdot (1 + \sum_{\ell \in L} d_\ell)} \cdot \text{runtime}_d^* \right) = \gamma \cdot \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right) \text{mul},$$

Proof. It follows straightforwardly from previous discussion and the Remarks 1 and 3. $\square$

5 Experimental Results

Since algorithms Searcher_d and SuperSearcher_d for $d \in \{2, 4\}$ perform on top of 2-isogenies (and 3-isogenies) between Montgomery curves, we generate “random” curve instances by walking through the 5-isogeny graph to avoid bias. We take $j_0 = 287496$ as the supersingular j -invariant origin and calculate a random 5-isogeny walk $j_0 \xrightarrow{\phi_1} j_1 \xrightarrow{\phi_2} \dots \xrightarrow{\phi_n} j_n$ of length $n = \log_5 p$. Next, we compute a Montgomery curve $\mathcal{E}: y^2 = x^3 + Ax^2 + x$ with j -invariant j_d and find a basis $\{P, Q\}$ of the torsion subgroup $\mathcal{E}[d^h]$ with $d \in \{2, 4, 12\}$. Given that $\#\mathbb{S}_p$ is unknown in practice, we take $h = \frac{1}{2} \log_2 p - \epsilon$ for some specific values of ϵ . Our experiments confirm our theoretical results, and highlight the improvements against [SCS22]. We provide a proof-of-concept implementation using SageMath [The25], available at [Com26].

Experiments concerning Searcher_d and SuperSearcher_d : We center our experiments on r -bits prime numbers of the form $p = 2^a \cdot f - 1$ with $a \approx r$ and $a \approx \frac{1}{2}r$. In particular, our experiments limit to $r \in [31, 50] \cap \mathbb{Z}$. We brute force search the interval $[31, 50] \cap \mathbb{Z}$ to get primes with $a \approx r$, and employ the extended Euclidean algorithm technique from [Cos20] for primes with $a \approx \frac{1}{2}r$. We include experiments for the algorithms Searcher_2 and SuperSearcher_2 for both cases $a \approx \frac{1}{2}r$ and $a \approx r$ (see Tables 6 and 7 of Appendix A.2). Given that Searcher_4 and SuperSearcher_4 implicitly ask for $4 \mid a$, we limit our experiments to $a \approx r$ for these 4-variant algorithms (see Table 8 of Appendix A.2). Additionally,

- For Searcher_2 , we take as inputs $\mathcal{E}$, $x(P)$, $x(Q)$, $x(P - Q)$, and $h = \lfloor \frac{1}{2} \log_2 p \rfloor - 1$. For Searcher_4 , we use $h = \lfloor \frac{1}{2} \log_4 p \rfloor$.
- For SuperSearcher_2 , we take as inputs $\mathcal{E}$, $x([2^4]P)$, $x([2^4]Q)$, $x([2^4](P - Q))$, and $h = \lfloor \frac{1}{2} \log_2 p \rfloor - 4$. For SuperSearcher_4 , we use $h = \lfloor \frac{1}{2} \log_4 p \rfloor - 2$.

Experiments concerning Searcher_{12} and $\text{SuperSearcher}_{12}$: We include experiments for primes of the form $p = 2^a \cdot 3^b \cdot f - 1$ (such primes are used, for example, in [DLRW24]); we assume $f \in \{5, 7, 11\}$. We also perform experiments for $p = 12^a \cdot f - 1$ with $a \gtrsim \frac{1}{2} \log_{12} p$ and f being a prime number (see Table 9 of Appendix A.2). We brute-force search the interval $[29, 50] \cap \mathbb{Z}$ for finding these kinds of primes. In addition,

- For Searcher_{12} , we take as inputs $\mathcal{E}$, $x(P)$, $x(Q)$, $x(P - Q)$, and $h = \lfloor \frac{1}{2} \log_{12} p \rfloor$.
- For $\text{SuperSearcher}_{12}$, we take as inputs $\mathcal{E}$, $x([12]P)$, $x([12]Q)$, $x([12](P - Q))$, and $h = \lfloor \frac{1}{2} \log_{12} p \rfloor - 1$.

We also include experiments for the algorithms Searcher_2 and SuperSearcher_2 for these primes (see Table 10 of Appendix A.2), and provide some additional graphs in Appendix A.3.

6 Concluding Remarks

Our results in Section 4, backed up with our experimental results from Section 5, highlight a time complexity of $\mathcal{O}(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p}) = \mathcal{O}(p^{1/2})$ field multiplications and memory complexity polynomial in $\log_2 p$ for both algorithms Searcher_d and SuperSearcher_d .¹⁰

Some remarks on the impact on current constructions. We emphasize that our results should be taken as cautionary advice for proposing conservative parameter sets in isogeny-based constructions with 2λ -bits primes of the form $p = 2^a \cdot f - 1$ with $a \geq \lambda$ and $\lambda \in \{128, 192, 256\}$. For instance, most isogeny-based constructions propose parameter sets that take into account the asymptotic bit complexity $\mathcal{O}((\log_2 p)^\kappa p^{1/2})$, without considering any constant factors and taking the value of $\kappa = 2$ [EHMP20]. This consideration is, for example, presented in SQIsign v2.0 [AAA⁺25, Section 10.2.1] and aligned to the NIST security considerations of [Nat22, Section 4.B.3], which we list and summarize below.

- NIST Security Level 1 asks for a classical complexity of 2^{143} bit operations.
- NIST Security Level 3 asks for a classical complexity of 2^{207} bit operations.
- NIST Security Level 5 asks for a classical complexity of 2^{272} bit operations.

However, if we consider Toom-Cook algorithm for the multiplication field elements in $\mathbb{F}_p$, then our results suggest a smaller value of $\kappa = \log_3 5 \approx 1.465$ compared to the state of the art, which implies a lower bit security than expected: 5.8, 2.9 and 6.3 bits below the NIST security requirements for Level 1, 3 and 5, respectively. We present in Table 5 some concrete numbers to illustrate the impact of our results from Section 4. Such a table provides asymptotic bit complexities for specific isogeny-based constructions.

Table 5: All numbers are given in $\log_2$ base. The 3rd column concern the quantities $(\log_2 p)^2 p^{1/2}$. The 4th and 5th columns consider Karatsuba and Toom-Cook (Toom-3) algorithms for the field multiplication; that is, they determine $(\log_2 p)^\kappa p^{1/2}$ with $\kappa \in \{\log_2 3, \log_3 5\}$, respectively. The 6th column consider the Schönhage-Strassen algorithm.

Scheme	$\log_2 p$	[EHMP20]	This work		
			Karatsuba	Toom Cook	Schönhage Strassen
SQIsign v1.0 [CSCD ⁺ 23]	254.0	143.0	139.7	138.7	139.6
	382.0	208.2	204.6	203.6	204.3
	510.0	273.0	269.3	268.2	268.8
SQIsignHD [DLRW24]	254.0	143.0	139.7	138.7	139.6
	389.0	211.7	208.1	207.1	207.8
	519.0	277.5	273.8	272.7	273.4
SQIsign2D-West [BDD ⁺ 24]	251.0	141.4	138.1	137.2	138.0
	383.0	208.7	205.1	204.1	204.8
	505.0	270.5	266.7	265.7	266.3
SQIPrime [DF24]	254.0	143.0	139.7	138.7	139.6
	413.0	223.9	220.3	219.2	220.0
	508.0	272.0	268.2	267.2	267.8
SQIsign2D-East [NOC ⁺ 24]	258.0	145.0	141.7	140.7	141.6
	386.0	210.2	206.6	205.6	206.3
	515.0	275.5	271.8	270.7	271.3
ApresSQI [SEMR24]	254.0	143.0	139.7	138.7	139.6
SQIsign v2.0 [AAA ⁺ 25]	251.0	141.4	138.1	137.2	138.0
	383.0	208.7	205.1	204.1	204.8
	505.0	270.5	266.7	265.7	266.3

¹⁰Our algorithms are highly parallelizable and extend to other primes (see Appendices A.4 and A.5).

Acknowledgements

We thank each anonymous reviewer for their valuable comments that helped to improve early versions of this manuscript. We additionally thank Loïc Bidoux, Mukul Kulkarni and Ricardo Pontaza-Rodas for their insightful comments and suggestions on this work.

References

- [AAA⁺25] Marius A. Aardal, Gora Adj, Diego F. Aranha, Andrea Basso, Isaac Andrés Canales Martínez, Jorge Chavez-Saab, Maria Corte-Real Santos, Pierrick Dartois, Luca De Feo, Max Duparc, Jonathan Komada Eriksen, Tako Boris Fouotsa, Décio Luiz Gazzoni Filho, Basil Hess, David Kohel, Antonin Leroux, Patrick Longa, Luciano Maino, Michael Meyer, Kohei Nakagawa, Hiroshi Onuki, Lorenz Panny, Sikhar Patranabis, Christophe Petit, Giacomo Pope, Krijn Reijnders, Damien Robert, Francisco Rodríguez-Henríquez, Sina Schaeffler, and Benjamin Wesolowski. *SQIsign. Tech. rep., National Institute of Standards and Technology*, 2025. Available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures>.
- [ACC⁺19] Gora Adj, Daniel Cervantes-Vázquez, Jesús-Javier Chi-Domínguez, Alfred Menezes, and Francisco Rodríguez-Henríquez. On the cost of computing isogenies between supersingular elliptic curves. In Carlos Cid and Michael J.: Jacobson, Jr., editors, *SAC 2018*, volume 11349 of *LNCS*, pages 322–343. Springer, Cham, August 2019. doi:10.1007/978-3-030-10970-7_15.
- [BDD⁺24] Andrea Basso, Pierrick Dartois, Luca De Feo, Antonin Leroux, Luciano Maino, Giacomo Pope, Damien Robert, and Benjamin Wesolowski. SQIsign2D-west - the fast, the small, and the safer. In Chung and Sasaki [CS24], pages 339–370. doi:10.1007/978-981-96-0891-1_11.
- [BDLS20] Daniel J. Bernstein, Luca De Feo, Antonin Leroux, and Benjamin Smith. Faster computation of isogenies of large prime degree. *ANTS XIV—Proceedings of the Fourteenth Algorithmic Number Theory Symposium*, 4(1):39–55, 2020. doi:10.2140/obs.2020.4.39.
- [CH17] Craig Costello and Hüseyin Hisil. A simple and compact algorithm for SIDH with arbitrary degree isogenies. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part II*, volume 10625 of *LNCS*, pages 303–329. Springer, Cham, December 2017. doi:10.1007/978-3-319-70697-9_11.
- [CLG09] Denis Xavier Charles, Kristin E. Lauter, and Eyal Z. Goren. Cryptographic hash functions from expander graphs. *Journal of Cryptology*, 22(1):93–113, January 2009. doi:10.1007/s00145-007-9002-x.
- [Com26] Companion Repository for this Manuscript. The code is provided as supplementary material, 2026.
- [Cos20] Craig Costello. B-SIDH: Supersingular isogeny Diffie-Hellman using twisted torsion. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part II*, volume 12492 of *LNCS*, pages 440–463. Springer, Cham, December 2020. doi:10.1007/978-3-030-64834-3_15.
- [CS24] Kai-Min Chung and Yu Sasaki, editors. *ASIACRYPT 2024, Part III*, volume 15486 of *LNCS*. Springer, Singapore, December 2024.

- [CSCD⁺23] Jorge Chavez-Saab, Maria Corte-Real Santos, Luca De Feo, Jonathan Komada Eriksen, Basil Hess, David Kohel, Antonin Leroux, Patrick Longa, Michael Meyer, Lorenz Panny, Sikhar Patranabis, Christophe Petit, Francisco Rodríguez-Henríquez, Sina Schaeffler, and Benjamin Wesolowski. *SQIsign. Tech. rep., National Institute of Standards and Technology*, 2023. Available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>.
- [DF24] Max Duparc and Tako Boris Fouotsa. SQIPrime: A dimension 2 variant of SQISignHD with non-smooth challenge isogenies. In Chung and Sasaki [CS24], pages 396–429. doi:10.1007/978-981-96-0891-1_13.
- [DG16] Christina Delfs and Steven D. Galbraith. Computing isogenies between supersingular elliptic curves over $\mathbb{F}_p$. *DCC*, 78(2):425–440, 2016. doi:10.1007/s10623-014-0010-1.
- [DLRW24] Pierrick Dartois, Antonin Leroux, Damien Robert, and Benjamin Wesolowski. SQISignHD: New dimensions in cryptography. In Joye and Leander [JL24], pages 3–32. doi:10.1007/978-3-031-58716-0_1.
- [EHMP20] Kirsten Eisenträger, Sean Hallgren, Chris Leonardi Travis Morrison, and Jennifer Park. Computing endomorphism rings of supersingular elliptic curves and connections to path-finding in isogeny graphs. *ANTS XIV—Proceedings of the Fourteenth Algorithmic Number Theory Symposium*, 4(1):215–232, 2020. doi:10.2140/obs.2020.4.215.
- [JL24] Marc Joye and Gregor Leander, editors. *EUROCRYPT 2024, Part I*, volume 14651 of *LNCs*. Springer, Cham, May 2024.
- [Koh96] David Kohel. *Endomorphism rings of elliptic curves over finite fields*. Phd thesis, University of California at Berkeley, 1996. Available at <https://www.i2m.univ-amu.fr/perso/david.kohel/pub/thesis.pdf>.
- [Mon87] Peter L. Montgomery. Speeding the pollard and elliptic curve methods of factorization. *Mathematics of computation*, 48(177):243–264, 1987.
- [Nat22] National Institute of Standards and Technology. *Call for Additional Digital Signature Schemes for the Post-Quantum Cryptography Standardization Process*, 2022. Available at <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/call-for-proposals-dig-sig-sept-2022.pdf>.
- [NOC⁺24] Kohei Nakagawa, Hiroshi Onuki, Wouter Castryck, Mingjie Chen, Riccardo Invernizzi, Gioella Lorenzon, and Frederik Vercauteren. SQIsign2D-east: A new signature scheme using 2-dimensional isogenies. In Chung and Sasaki [CS24], pages 272–303. doi:10.1007/978-981-96-0891-1_9.
- [Ren18] Joost Renes. Computing isogenies between Montgomery curves using the action of $(0, 0)$. In Tanja Lange and Rainer Steinwandt, editors, *Post-Quantum Cryptography - 9th International Conference, PQCrypto 2018*, pages 229–247. Springer, Cham, 2018. doi:10.1007/978-3-319-79063-3_11.
- [Sco20] Michael Scott. A note on the calculation of some functions in finite fields: Tricks of the trade. Cryptology ePrint Archive, Report 2020/1497, 2020. URL: <https://eprint.iacr.org/2020/1497>.

-
- [SCS22] Maria Corte-Real Santos, Craig Costello, and Jia Shi. Accelerating the delfsgalbraith algorithm with fast subfield root detection. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 285–314. Springer, Cham, August 2022. doi:[10.1007/978-3-031-15982-4_10](https://doi.org/10.1007/978-3-031-15982-4_10).
- [SEMR24] Maria Corte-Real Santos, Jonathan Komada Eriksen, Michael Meyer, and Krijn Reijnders. AprèsSQI: Extra fast verification for SQIsign using extension-field signing. In Joye and Leander [JL24], pages 63–93. doi:[10.1007/978-3-031-58716-0_3](https://doi.org/10.1007/978-3-031-58716-0_3).
- [The25] The Sage Developers. *SageMath, the Sage Mathematics Software System (Version 10.7)*, 2025. <https://www.sagemath.org>.

A Appendix

A.1 On the small tweak on Searcher_4 and SuperSearcher_4

Let us first focus on the algorithm Searcher_4 . Recall that the maximum number of subfield curves (that is, the number of visited nodes in the 4-isogeny tree) is approximately $\frac{4}{3} \cdot 4^h$ where $h = \log_4 \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right)$.

Let us follow the notation from Section 4, and let $\mathcal{E}_b$ a node in the 4-isogeny tree rooted at (some given) supersingular curve $\mathcal{E}$. Observe that the optimized 4-isogeny formulas of [Ren18] exclude the explicit calculation of the intermediate 2-isogenies located between $\mathcal{E}_b$ and $\mathcal{E}_{b_j}$ for $j := 0, 1, 2, 3$. In particular, such intermediate 2-isogeny has as kernel generator the order-2 point $(0, 0)$ [Ren18].

There are two main remarks about such intermediate 2-isogenies:

1. The 4-isogenies $\phi_{b0}: \mathcal{E}_b \rightarrow \mathcal{E}_{b0}$ and $\phi_{b2}: \mathcal{E}_b \rightarrow \mathcal{E}_{b2}$ pass through the same intermediate 2-isogenous curve, and
2. The 4-isogenies $\phi_{b1}: \mathcal{E}_b \rightarrow \mathcal{E}_{b1}$ and $\phi_{b3}: \mathcal{E}_b \rightarrow \mathcal{E}_{b3}$ pass through the same intermediate 2-isogenous curve.

Then, the inspection of the intermediate curves through 2-isogenies with kernel $\langle (0, 0) \rangle$ includes

$$\sum_{k=0}^{h-1} 2 \cdot 4^k = \frac{2}{3} \cdot (4^h - 1) \approx \frac{2}{3} \cdot 4^h$$

extra subfield checks. Hence, we would inspect a total of $2 \cdot 4^h$ curves. Now, if $\mathcal{E}_{b_j}: y^2 = x^3 + Ax^2 + x$ for some $A = A'/C \in \mathbb{F}_{p^2}$ then the j-invariant of the intermediate 2-isogenous curves is

$$j = \frac{16 \cdot (A^2 + 12)^3}{(A^2 - 4)^2} = \frac{16 \cdot (A'^2 + 12C^2)^3}{C^2 \cdot (A'^2 - 4C)^2}.$$

Therefore, checking if $j \in \mathbb{F}_p$ requires $2M + 4S + 2\text{mul} = 14\text{mul}$ field operations when A' and C' are given. Hence, the inspection of the intermediate curves through 2-isogenies with kernel $\langle (0, 0) \rangle$ includes

$$\sum_{k=0}^{h-1} 2 \cdot 14 \cdot 4^k = \frac{28}{3} \cdot (4^h - 1) \approx \frac{28}{3} \cdot 4^h < 10 \cdot 4^h.$$

extra field operations over $\mathbb{F}_p$. Consequently, the total running time of Searcher_4 is now bounded by $92 \cdot \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right)$ field operations, while its average running time reduces to $23 \cdot \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right)$ field operations.

Let us now discuss the impact on the algorithm SuperSearcher_4 . In this scenario, it is just a matter of replacing 82 by 92 in the analysis of the running time of SuperSearcher_4 (see Section 4.3). In particular, the running time of SuperSearcher_4 changes to be bounded by $62.5 \cdot \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right)$, while its average running time becomes $28.85 \cdot \left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p} \right)$.

Hence, adding the 2-isogeny inspection only helps to slightly improve the average running time of Searcher_4 . In particular, it does not help to improve the algorithm SuperSearcher_4 .

A.2 Tables concerning the experiments

We provide all tables concerning our experiments in Section 5.

Table 6: All the numbers correspond with the average of solving 100 random instances by using **Searcher₂** and **SuperSearcher₂**. The Avg. γ values present the measured average value of γ ; i.e., the measured average running time of each algorithm is $\gamma \cdot 2^n$ with $n = \frac{1}{2} \log_2 p$. For **SuperSolver**, the value of γ includes the logarithm factor $\log_2 p$. The rates determine the success ratio of each experiment.

Parameters										
$\log_2 p$:	32	35	37	38	40	41	42	43	50	
a :	17	21	19	19	22	21	22	22	26	
f :	19417	10645	140995	337825	152945	601135	861533	1924415	9365135	
SuperSolver										
Avg. γ :	19.0	32.0	15.1	19.3	5.5	7.9	13.8	18.1	12.5	
Rate:	0.9	0.7	1.0	1.0	1.0	1.0	1.0	0.9	1.0	
Searcher₂										
Avg. γ :	20.2	25.7	30.8	21.4	14.0	21.7	22.2	24.3	15.8	
Rate:	1.0	1.0	0.8	1.0	1.0	0.9	1.0	0.8	1.0	
SuperSearcher₂										
Avg. γ :	20.5	18.4	21.4	19.0	8.5	17.9	18.7	23.2	13.2	
Rate:	0.9	1.0	1.0	0.9	1.0	1.0	1.0	0.9	1.0	

Table 7: The description is the same as in Table 6.

Parameters												
$\log_2 p$:	31	35	36	37	38	40	41	42	45	46	47	48
a :	31	32	34	31	33	35	36	37	39	41	40	42
f :	1	5	3	43	31	31	17	31	61	19	107	53
SuperSolver												
Avg. γ :	18.1	12.7	6.1	33.9	14.2	24.6	23.6	16.4	15.7	19.8	23.8	20.5
Rate:	1.0	1.0	1.0	0.6	1.0	0.8	0.9	1.0	0.9	0.8	0.8	1.0
Searcher₂												
Avg. γ :	21.4	23.4	9.3	40.3	24.8	31.6	29.9	21.5	40.6	16.9	25.1	29.2
Rate:	1.0	0.9	1.0	0.7	0.9	0.8	0.8	0.9	0.8	1.0	1.0	0.7
SuperSearcher₂												
Avg. γ :	18.6	25.5	9.8	31.4	15.9	22.0	14.5	20.3	26.7	17.6	16.2	18.0
Rate:	0.9	0.8	1.0	0.9	1.0	1.0	1.0	1.0	0.8	1.0	1.0	1.0

Table 8: The description is the same as in Table 6, with the difference that these experiments correspond to algorithms **Searcher₄** and **SuperSearcher₄**.

Parameters												
$\log_2 p$:	31	35	36	37	38	40	41	42	45	46	47	48
a :	31	32	34	31	33	35	36	37	39	41	40	42
f :	1	5	3	43	31	31	17	31	61	19	107	53
SuperSolver												
Avg. γ :	18.1	12.7	6.1	33.9	14.2	24.6	23.6	16.4	15.7	19.8	23.8	20.5
Rate:	1.0	1.0	1.0	0.6	1.0	0.8	0.9	1.0	0.9	0.8	0.8	1.0
Searcher₄												
Avg. γ :	14.4	16.0	5.0	37.4	18.5	28.2	19.0	8.7	33.1	7.8	16.6	16.9
Rate:	1.0	1.0	1.0	0.9	0.9	0.9	1.0	1.0	0.9	1.0	1.0	1.0
SuperSearcher₄												
Avg. γ :	11.0	11.9	5.2	18.2	8.7	15.7	10.4	6.9	20.8	10.0	9.4	11.6
Rate:	1.0	1.0	1.0	0.9	0.9	1.0	1.0	0.9	0.8	0.9	1.0	1.0

A.3 Plots concerning the experiments

We graphically illustrate the comparison between our proposed algorithms in Figures 6a to 6f and 7a to 7d. The horizontal line in **black** ink determines the constant from Equation (6). The dashed lines in **orange** ink describe the minimum and maximum values obtained from 25 random experiments, while the **blue** dashed lines refer to the first and third quartiles. The line in **blue** ink between the quartiles concerns the Median measured values.

Table 9: The description is the same as in Table 6, with the difference that these experiments correspond to algorithms Searcher_{12} and $\text{SuperSearcher}_{12}$.

Parameters								
$\log_2 p$:	30	34	36	40	42	44	48	50
a :	4	5	5	6	6	6	7	7
f :	32783	65599	262151	262231	1048793	4194493	4194329	16777499
SuperSolver								
Avg. γ :	5.3	6.1	5.9	12.7	7.4	6.5	4.1	7.6
Rate:	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Searcher₁₂								
Avg. γ :	18.9	28.0	12.6	17.8	12.5	18.7	12.7	10.1
Rate:	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
SuperSearcher₁₂								
Avg. γ :	4.9	7.4	6.5	11.3	7.6	6.9	6.5	4.0
Rate:	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0

Table 10: The description is the same as in Table 6.

Parameters							
$\log_2 p$:	29	34	36	37	38	39	45
a :	15	18	19	19	19	20	24
f :	15309	45927	98415	216513	413343	413343	1240029
SuperSolver							
Avg. γ :	19.5	13.4	6.5	14.2	12.6	12.0	12.5
Rate:	0.9	1.0	1.0	1.0	1.0	1.0	1.0
Searcher₂							
Avg. γ :	16.3	15.1	10.7	22.5	9.2	21.3	28.0
Rate:	1.0	1.0	1.0	1.0	1.0	1.0	0.9
SuperSearcher₂							
Avg. γ :	10.9	13.4	11.5	15.0	14.3	27.0	18.1
Rate:	1.0	1.0	1.0	1.0	1.0	0.8	1.0

A.4 Parallel search optimizations.

Observe that the presented subfield curve searching algorithms are highly parallelizable when having $M = 2^m$ processors: one can precompute all the nodes at depth m , and each processor takes one of those nodes as the root and inspects a subtree of height $(d - m)$.

In what follows, we describe another interesting parallel search. Let us assume $M \in \mathbb{N}$, take an integer ℓ of $m = \lfloor \log_2 M \rfloor$ bits, and proceed as below.

- Find a basis $\{R, S\}$ for $\mathcal{E}[\ell]$.
- For the k^{th} processor:
 - Calculate the ℓ -isogeny $\psi_k: \mathcal{E} \rightarrow \mathcal{E}_k$ with kernel $\langle R + [k]S \rangle$.
 - Run the subfield curve searching algorithm with inputs $\mathcal{E}_k$, $\psi_k(P)$, $\psi_k(Q)$, and $h - m$.

The main limitation of the above parallel search is that we require ℓ to factorize as the product of “small” primes $\ell_1, \dots, \ell_r$. But, for efficiency, most of the isogeny-based cryptographic constructions assume $(p+1)(p-1) = p^2 - 1$ is as smooth as possible (which favors this kind of parallel search). Therefore, one can take ℓ that divides $(p^2 - 1)$ and satisfies $\ell \approx M$.¹¹ Notice that the cost of computing ψ_k is negligible compared with the running time of our subfield curve searching algorithms when $m \ll \frac{1}{2} \log_2 p$, and the number of processors is not limited to a power of two.

¹¹For the case of SuperSearcher_2 and RadicalSearcher_2 , one should assume ℓ is not divisible by some integer in the input list L .

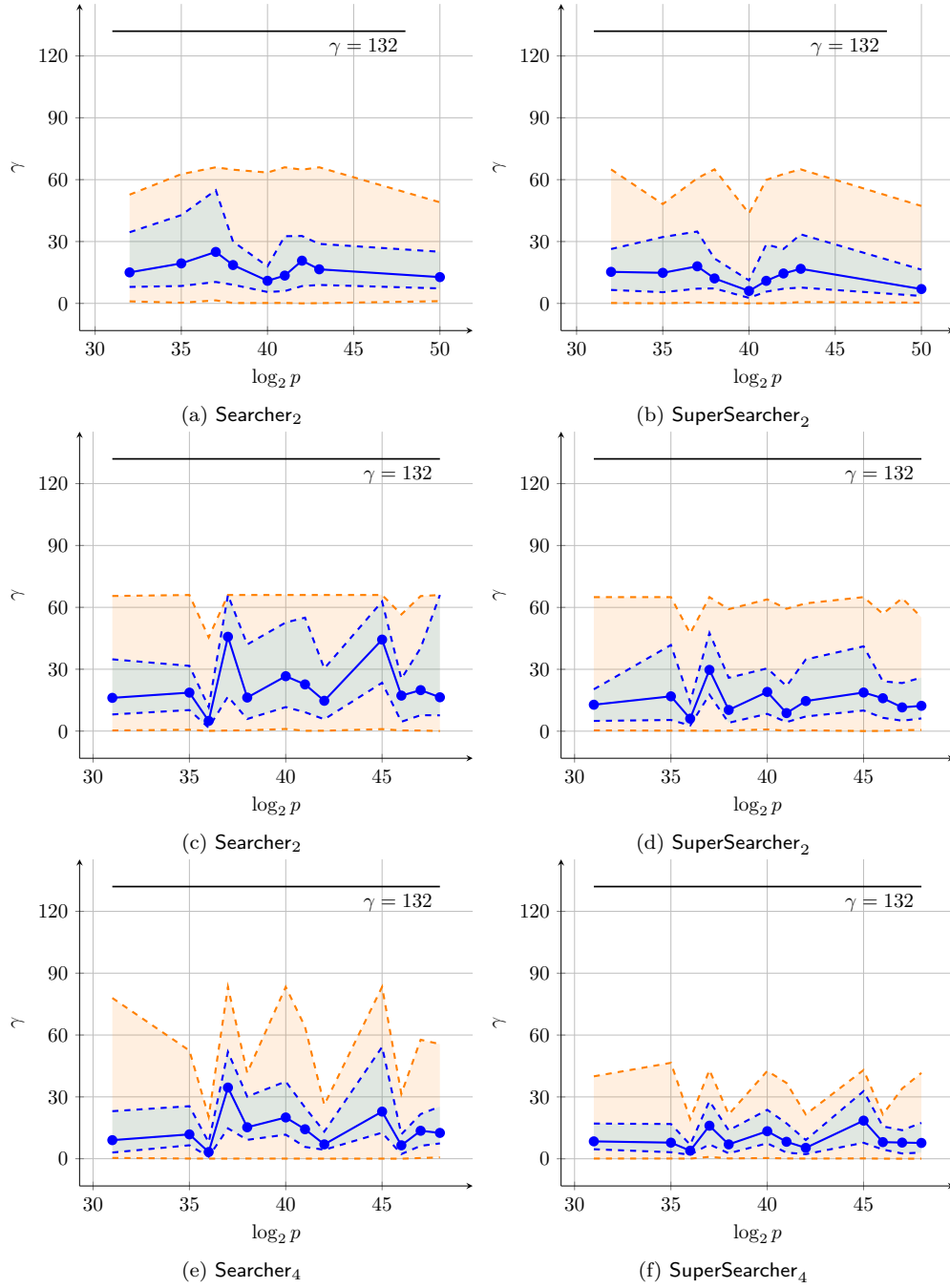


Figure 6: Experiments in Figures 6a and 6b correspond with the prime numbers listed in Table 6. Experiments in Figures 6c and 6d correspond with the prime numbers listed in Table 7. Experiments in Figures 6e and 6f correspond with the prime numbers listed in Table 8.

Certainly, the calendar time when applying the above parallel search techniques becomes $\mathcal{O}(p^{1/2}/M)$ but the total running time is still $\mathcal{O}(p^{1/2})$.¹²

¹²By calendar time, we mean the elapsed time taken for a computation. The total time is the sum of the time expended by all M processors.

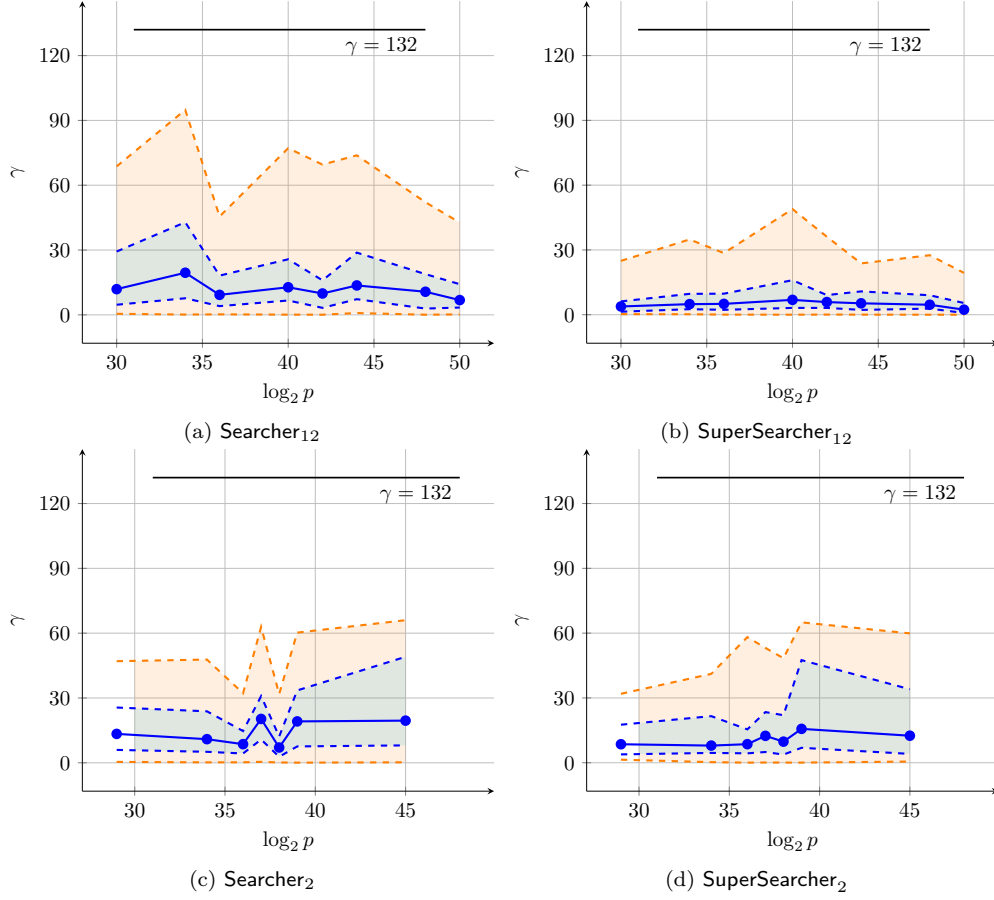


Figure 7: Experiments in Figures 7a and 7b correspond with the prime numbers listed in Table 9. Experiments in Figures 7c and 7d correspond with the prime numbers listed in Table 10.

A.5 Subfield Curve Searching Over More Generic Fields

At this point, we have provided subfield curve search algorithms with time complexity of

$$\mathcal{O}\left(\frac{\#\mathbb{S}_{p^2}}{\#\mathbb{S}_p}\right) = \mathcal{O}(p^{1/2}) \text{ mul} \quad (10)$$

operations, for primes of the form $p = d^a \cdot f - 1$ for small $d \in \mathbb{N}$ and some $a, f \in \mathbb{N}$. Certainly, the condition in the form of p limits the applicability of the aforementioned algorithms Searcher_d and SuperSearcher_d . However, we next discuss how to extend our current results to more generic primes.

Applicability to primes of the form $p = 2^a \cdot f + 1$. We emphasize that these algorithms straightforwardly apply to primes of the form $p = 2^a \cdot f + 1$. The reasoning for this claim relies on two facts: The torsion subgroup $\mathcal{E}[2^a]$ has two generators, P and Q , with x -coordinates in the field $\mathbb{F}_{p^2}$ and y -coordinates in $\mathbb{F}_{p^4}$ [Cos20], and the optimized Algorithms from Section 4 perform operations on the Kummer line (i.e., we only operate with the x -coordinates). The main difference when employing primes of the form $p = 2^a \cdot f + 1$ is that we work on a field extension $\mathbb{F}_{p^2} = \mathbb{F}_p[x]/(x^2 + \mu)$ with a small $\mu \in \mathbb{F}_p$ different from 1. That difference does not affect the asymptotic complexity of the

algorithms (e.g., at the level of operations in $\mathbb{F}_{p^2}$, we still perform the same number of multiplications and squares).

Extending to other cryptographic interest primes. Notice that algorithm Searcher_d can be extended to the setting $p = 4 \cdot d_1 \cdot d_2 \cdots d_n - 1$ for some small odd primes d_i 's. The main difference for that scenario relies on the isogeny step, which requires performing d_i -isogenies instead of d -isogenies. Each d_i -isogeny has a small degree $d_i \ll p^{1/2}$ and a computational cost of $\tilde{O}(\sqrt{d_i})$ mul operations [BDLS20]. A similar situation occurs for a prime p such that $(p-1)$ and $(p+1)$ are B -smooth for some $B \in \mathbb{N}$ as in [Cos20, CSCD⁺23].

In both cases, the torsion subgroup $\mathcal{E}[p-1]$ has two generators, P and Q , with x -coordinates in the field $\mathbb{F}_{p^2}$ and y -coordinates in $\mathbb{F}_{p^4}$ [Cos20]. Hence, we can provide an algorithm that relies only on the x -coordinates and still allows a time complexity of $\mathcal{O}(p^{1/2})$ field multiplications in $\mathbb{F}_{p^2}$. The core idea relies on considering m -isogenies with $m \mid (p^2 - 1)$ such that $m = d_1 \cdot d_2 \cdots d_h \approx p^{1/2}$, $d_1 \leq d_2 \leq \cdots \leq d_h$ and the list $M = \{d_1, d_2, \dots, d_h\}$ includes the h smallest prime factors of $(p^2 - 1)$. Based on that, our proposed algorithm proceeds by computing d_i -isogenies going from d_h to d_1 . Calculating the isogenies in that ordering ensures that the largest d -isogenies are performed the least number of times, while the smallest isogenies are computed as much as possible (e.g. an exponential number of times); thus, the running time is expected to be as in Equation (10) since the smaller isogenies have a running time close to $\mathcal{O}(1)$. We label Searcher_M to distinguish it from Searcher_d , and summarize the main difference on this setting below.

Setup when employing d_{n-1} -isogenies. Following the notation from Fig. 3, the input must satisfy $\mathcal{E}[m] = \langle P, Q \rangle$. In addition, the integer n determines the length of the sequence $\mathbf{b} \in \{0, \dots, d_n - 1\} \times \{0, \dots, d_{n-1} - 1\} \times \cdots \times \{0, \dots, d_1 - 1\}$, and we denote $\mathbf{z}_{\mathbf{b}} = \{\mathcal{E}, \mathbf{b}, x(P_{\mathbf{b}}), x(Q_{\mathbf{b}}), x(R_{\mathbf{b}})\}$ where $R_{\mathbf{b}} = P_{\mathbf{b}} - Q_{\mathbf{b}}$.

Calculating Codomain d_{n-1} -isogenies when $n < (h-1)$. We need to calculate d_{n-1} different d_{n-1} -isogenies:

$$\phi_{\mathbf{b}_j} : \mathcal{E}_{\mathbf{b}} \rightarrow \mathcal{E}_{\mathbf{b}_j} \text{ with } \ker \phi_{\mathbf{b}_j} = \langle [\prod_{i=1}^{n-1} d_i](P_{\mathbf{b}} + [j]Q_{\mathbf{b}}) \rangle \text{ for each } j := 0, \dots, d_{n-1} - 1,$$

Calculating Codomain d_1 -isogenies when $n = (h-1)$. We need to calculate d_1 different d_1 -isogenies:

$$\phi_{\mathbf{b}_j} : \mathcal{E}_{\mathbf{b}} \rightarrow \mathcal{E}_{\mathbf{b}_j} \text{ with } \ker \phi_{\mathbf{b}_j} = \langle P_{\mathbf{b}} + [j]Q_{\mathbf{b}} \rangle \text{ for each } j := 0, \dots, d_1 - 1.$$

Pushing Points Through d_{n-1} -isogenies. We need to push a total of $3d_{n-1}$ x -only points through the isogenies:

$$\begin{aligned} x(P_{\mathbf{b}_j}) &= x(\phi_{\mathbf{b}_j}(P_{\mathbf{b}} + [j]Q_{\mathbf{b}})), & x(Q_{\mathbf{b}_j}) &= x(\phi_{\mathbf{b}_j}([d_{n-1}]Q_{\mathbf{b}})), \text{ and} \\ x(R_{\mathbf{b}_j}) &= x(\phi_{\mathbf{b}_j}(P_{\mathbf{b}} - [d_{n-1} - j]Q_{\mathbf{b}})) \end{aligned}$$

for each $j := 0, \dots, d_{n-1} - 1$.

Remark 4. Since the subfield curve detection algorithm in Section 4.3 exploits inspection through small d -isogenies, it suggests that a variant of SuperSearcher_M must require to inspect for values of $\ell \notin M$. In particular, for primes of the form $p = 4 \cdot d_1 \cdot d_2 \cdots d_n - 1$, we expect that SuperSearcher_M coincides with Searcher_M (i.e., $L = \emptyset$). A similar situation should occur for primes p such that $(p-1)$ and $(p+1)$ are B -smooth.